

从 C++ 到计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册已经回答了基础问题：一个 C++ 程序如何变成进程、地址空间、二进制接口、汇编指令和可测量的执行现象。它负责打牢源码到二进制、x86-64 汇编、System V ABI、Linux 基础工具、虚拟地址空间和可信 benchmark 的底座。第二册不再把这些内容重新讲一遍，而是把它们当作读者已经掌握的工具：能看汇编，能确认二进制身份，能写基本实验报告，能理解进程和虚拟地址空间的基本边界。

本册继续向下和向外展开，但不提前进入 AI。真正要理解高性能程序，必须先把“计算本身”讲清楚：核心怎样取指、译码、调度和提交，乱序执行为什么能隐藏延迟，分支预测为什么会失败，缓存和 TLB 为什么决定很多程序的上限，多核心之间为什么需要一致性协议，NUMA 为什么让同一块内存存在不同核心看来成本不同。第一册把单个程序放到机器上，第二册追问这台机器怎样扩展为多核心计算系统，再怎样扩展为多进程、多节点和分布式计算系统。

软件层面同样不能跳步。多线程不是简单地把循环切成几份；锁不是只有“慢”一个性质；原子操作不是更快的锁；无锁数据结构也不是不用同步。并行算法要面对数据依赖、负载均衡、局部性、归约顺序和任务粒度。分布式计算更进一步：一旦机器之间只能通过网络通信，延迟、带宽、故障、重试、超时、复制和一致性就会成为计算模型的一部分。

因此第二册的主题是计算系统，而不是某个应用领域。它从硬件原理开始，讲到单机高性能计算、多线程并发、锁与无锁结构、并行算法、运行时系统、异步 I/O 和分布式计算。AI 模型、张量、算子开发和量化推理引擎会放到第三册。这样安排不是延后重点，而是先把地基打牢：没有硬件和并发基础，后续任何 AI 算子优化都会变成经验堆砌。

核心思想

第二册的核心问题是：给定真实硬件、操作系统和网络边界，怎样组织数据、线程、同步、任务和通信，使计算能够正确、可测量、可扩展地运行。

本册仍然沿用第一册的写法：原理先行，代码作为证据；实验必须记录输入、环境、命令和误差；源码阅读抓数据结构、热路径、同步边界和性能瓶颈。不过，本册的证据重心会改变。第一册主要用反汇编、ELF、GDB、`readelf`、`objdump` 和基础 `perfstat` 建立“这段程序真的这样执行”的证据；第二册会更多使用拓扑、PMU、调度事件、锁等待、cache line 迁移、NUMA 放置、队列长度、吞吐曲线和失败注入来解释“系统为什么不能继续扩展”。

读者读完后，应当能解释一个多核程序为什么没有线性加速，能判断锁竞争和 false sharing 的差异，能写出可测试的并行归约、有界队列和环形队列，能读懂 Linux 调度、futex、RCU、内存管理和 perf 相关源码入口，也能把单机计算原则扩展到分布式系统。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。普通 x86-64 Linux 台式机、笔记本或服务器适合完成完整实验；Termux 可以用于阅读、构建小实验和运行基础测试，但 perf、NUMA、线程亲和性、硬件计数器、futex 观察、eBPF 观测和分布式网络实验可能受权限、内核和设备能力限制。所有性能结论都必须写清环境。

阅读本册前，应默认已经完成第一册的最低要求：能把源码编译成二进制并确认产物身份，能读基本 x86-64 汇编，能解释调用约定和栈帧，能写出隔离热路径的 benchmark，能使用 Linux 常见工具观察进程、地址空间和基础性能计数。第二册不会反复解释这些背景；遇到这些内容时，本册只会说明它们怎样服务多核、同步、运行时或分布式问题。

阅读顺序建议严格按章节推进。前四章讲硬件执行模型，这是后面所有内容的解释基础。单机高性能计算部分讲测量、数据布局、SIMD 和典型内核，帮助读者把“程序慢”分解成计算、内存、分支、同步或 I/O 问题。并发部分讲线程、锁、条件变量、原子、内存序和无锁数据结构，重点不是背 API，而是理解共享状态怎样被保护、发布和回收。并行算法和分布式计算部分再把这些原则扩展到任务图、运行时、网络通信、复制和一致性。

本册的贯穿实验叫 Compute Systems Labs。它不追求一开始很大，而是从小而真实的模块开始：顺序归约、并行归约、带锁计数器、原子计数器、有界队列、任务切分和简单 benchmark。后续扩写会继续加入线程池、工作窃取、SPSC 环形队列、MPMC 队列、并行扫描、分布式 shuffle 和故障注入。

心智模型

读本册时始终追问四件事：数据在哪里，谁会读写它，硬件会怎样移动它，失败或竞争发生时系统怎样保持正确。

每章至少做一个实验。硬件章节要配合 `lscpu`、`numactl`、`perfstat`、拓扑图和内存访问 microbenchmark；并发章节要写出可重复触发竞争、阻塞、false sharing、CAS 失败或内存序问题的小程序；运行时章节要记录队列长度、任务粒度、worker 空闲时间和背压；分布式章节即使只在本机多进程模拟，也要记录延迟、重试、超时、重复请求和消息顺序。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求，性能来自数据复用、负载均衡、同步控制和测量证据。没有这些证据，加线程常常只是更快地制造争用。

本册的章节之间有明确承接关系。第一部分回答硬件成本从哪里来，第二部分回答怎样把成本变成可测假设，第三部分回答共享内存里怎样保持正确，第四部分回答怎样把多个线程组织成可扩展运行时，第五部分回答当共享内存消失、网络和故障进入模型后，计算原则怎样变化。不要把某章当成孤立技巧阅读；每次做实验，都要把结论回连到这条链上。

全书结构

本册分为五个部分，围绕“计算系统如何正确且高效地扩展”展开。第一册已经承担源码、进程、虚拟地址、汇编、ABI、工具链和基础测量，本册只在需要建立新成本模型时短暂回扣这些内容，不再重复铺陈。这样处理的目的，是把篇幅留给第一册没有充分展开的主题：多核心硬件、缓存一致性、NUMA、SIMD 和内存带宽、Linux 调度和同步、无锁结构、并行运行时、异步 I/O、分布式计算和故障模型。

第一部分是硬件执行模型。它讲核心流水线、乱序执行、分支预测、寄存器重命名、缓存层级、TLB、页表、预取、NUMA、互连和缓存一致性。读者要在这里建立硬件成本模型：一条指令不只是语法，背后有取指、依赖、执行端口、访存、提交和一致性流量。

第二部分是单机高性能计算。第一册已经讲过 benchmark 纪律，所以本部分不再停留在“怎样计时”这类基础问题，而是进入多核进阶测量、Roofline、PMU 归因、cache 和 TLB 事件、数据布局、局部性、SIMD、指令级并行和典型计算内核。目标是把“优化”从口号变成可验证的分析：到底是算力不够、内存带宽不够、cache miss 太多、分支预测差、TLB 覆盖不足、同步等待太长，还是 benchmark 边界仍然写错。

第三部分是多线程、同步与内存模型。它讲线程生命周期、调度、亲和性、锁、条件变量、信号量、原子操作、内存序、false sharing、无锁队列和内存回收。这里的重点是正确性先于性能：如果可见性、互斥、进度保证和对象生命周期没有说清楚，所谓高性能就是不稳定的。

第四部分是并行算法与运行时。归约、扫描、分区、图遍历、任务图、线程池、工作窃取、异步 I/O 和背压，都是把多核硬件变成可用计算能力的桥梁。读者在这里要学会从算法依赖和数据移动角度设计并程序序。

第五部分是分布式计算。它把单机原则扩展到网络和集群：RPC、序列化、时间、故障、分片、复制、一致性、共识、shuffle、检查点和观测性。分布式不是“多几台机器”，而是把通信和失败纳入计算模型。

核心思想

第二册的主线是一条从硬件到集群的链：硬件决定局部成本，多线程决定共享内存内的协作，并行算法决定可扩展形状，分布式系统决定跨机器的通信和故障边界。

和第一册的边界

第一册已经讲清楚的内容，本册只作为前提使用：源码到目标文件和 ELF 的路径，进程和虚拟地址空间的基本概念，x86-64 汇编阅读，System V AMD64 ABI，C++ 与汇编互操作，基础 Linux 命令，基础 benchmark 设计，基础 **perfstat** 和 Linux 源码入口。第二册遇到这些内容时，重点不是重复定义，而是解释它们在可扩展计算中的新作用。

例如，第一册讲虚拟地址空间时，目标是让读者理解进程地址、映射和缺页；第二册讲 TLB 和页级性能时，目标是解释页大小、page walk、NUMA 放置和首次触页怎样改变吞吐。第一册讲 **perfstat** 时，目标是建立可信测量意识；第二册讲 PMU 时，目标是用事件组合区分前端、后端、内存、同步和调度瓶颈。第一册讲汇编控制流时，目标是读懂分支和跳转；第二册讲分支预测时，目标是理解输入分布、预测器状态和流水线回滚怎样限制单核吞吐。

本册扩展的重点

本册扩展不是简单增加术语，而是围绕可扩展计算的核心矛盾展开。硬件部分要能从 pipeline、ROB、load/store queue、store buffer、cache line、TLB、NUMA 和一致性协议解释成本。同步部分要能从不变量、等待谓词、futex、内存序、线性化点和内存回收解释正确性。运行时部分要能从任务粒度、局部性、工作窃取、背压和关闭语义解释工程稳定性。分布式部分要能从消息、重试、幂等、复制、共识、shuffle、检查点和观测性解释跨机器计算。

每个章节都要有三个出口：能画出机制图，能写出最小可运行例子，能设计一份可复查实验。只

有这样，第二册才不是第一册的复述，而是把第一册的基础能力推进到现代计算系统。

目录

第一部分	硬件执行模型：计算为什么会快也会慢	
第 1 章	计算系统全景	2
第 2 章	核心流水线、乱序执行与分支预测	5
第 3 章	缓存、TLB 与页级性能	8
第 4 章	NUMA、互连与缓存一致性	12
第二部分	单机高性能计算：从测量到数据流	
第 5 章	可信测量、Roofline 与性能计数器	16
第 6 章	数据布局、局部性与预取	19
第 7 章	SIMD、指令级并行与向量化	22
第 8 章	典型计算内核模式	24
第三部分	多线程、同步与内存模型	
第 9 章	线程、调度与亲和性	27
第 10 章	锁、条件变量与信号量	29
第 11 章	原子操作与内存序	32
第 12 章	无锁数据结构	35
第四部分	并行算法与运行时	
第 13 章	归约、扫描与分区	39
第 14 章	任务运行时与工作窃取	41
第 15 章	I/O、异步与背压	43
第五部分	分布式计算：把单机原则扩展到集群	
第 16 章	分布式计算模型	46
第 17 章	分片、复制与共识	48
第 18 章	分布式计算工程实践	50
附录 A	实验路线	52
附录 B	源码阅读索引	53

第零部分

硬件执行模型：计算为什么会快也会慢

第 1 章

计算系统全景

第一册已经说明程序如何变成进程、虚拟地址、调用约定和机器指令。第二册继续追问：当一个程序需要利用多个核心、多个内存层级、多个线程甚至多台机器时，计算能力从哪里来，又会在哪里损失。这个问题不能只从代码出发，也不能只从硬件手册出发。真正的计算系统由硬件、操作系统、运行时、算法和通信协议共同构成。

1.1 从第一册到第二册

第一册的学习对象是一段程序如何成为一个可观察的执行实例。读者已经知道源文件、目标文件、链接、ELF、进程、虚拟地址、ABI、汇编指令和基础测量之间的关系。第二册的学习对象不是再把这条路径复述一遍，而是把这个执行实例放进更复杂的资源环境：多个核心同时执行，多个缓存层级同时保存数据，多个线程共享状态，多个任务争用队列，多个节点通过网络交换消息。

这意味着本册每次遇到第一册知识时，都要换一个问题问。看到汇编，不再只问“这条指令做什么”，还要问“它处在依赖链上还是可以并行发射”；看到虚拟地址，不再只问“它属于哪个映射”，还要问“这个访问是否会击穿 TLB、是否发生首次触页、页面是否放在远端 NUMA 节点”；看到 `perfstat`，不再只问“数字如何记录”，还要问“这些事件能否区分前端、后端、内存带宽、锁等待和调度噪声”。

核心思想

第一册把程序执行讲清楚，第二册把可扩展计算讲清楚。前者关心“程序如何被机器执行”，后者关心“多个执行实体怎样在硬件、操作系统和网络边界内协同前进”。

1.2 从一条指令到一个系统

一条加法指令看起来很简单，但它并不是孤立执行的。核心要取指、译码、重命名寄存器、等待操作数、选择执行端口、写回结果并提交状态。如果操作数在寄存器里，成本很低；如果来自 L1 cache，延迟增加；如果来自 L3、远端 NUMA 节点或磁盘，程序看到的时间会跨越几个数量级。并发程序还要考虑另一个核心是否正在写同一条 cache line，分布式程序还要考虑消息是否跨越网络。

因此本册把计算系统分成五层。第一层是硬件执行，包括核心、缓存、TLB、内存控制器和互连。第二层是操作系统，包括调度、页表、系统调用、I/O 和计数器。第三层是运行时，包括线程池、任务队列、同步原语和内存分配器。第四层是算法，包括归约、扫描、分区、图遍历和负载均衡。第五层是分布式协议，包括 RPC、复制、共识、shuffle 和检查点。

核心思想

高性能不是某一层单独决定的。硬件给出能力边界，操作系统分配资源，运行时组织执行，算法决定数据依赖，分布式协议处理通信和故障。

1.3 四种尺度

计算系统至少有四种尺度。第一种是单条指令尺度，讨论延迟、吞吐、依赖、端口和分支预测。第二种是单个核心尺度，讨论流水线、乱序窗口、寄存器重命名、load/store queue 和 SIMD。第三种是单台机器尺度，讨论缓存层级、TLB、NUMA、线程调度、锁、原子、队列和 I/O。第四种是集群尺度，讨论 RPC、分片、复制、共识、shuffle、检查点和故障恢复。

这四种尺度不能混用。单条指令尺度上，减少一条指令可能有效；单机多线程尺度上，减少一条指令可能完全被锁等待淹没；集群尺度上，减少一百万条指令也可能抵不过一次跨机重试。工程判断必须先确认当前瓶颈属于哪个尺度。第二册的核心训练之一，就是把“慢”定位到正确尺度，再选择对应证据。


```

source code
-> instructions and dependencies
-> core pipeline and local cache
-> shared-memory multicore machine
-> runtime queues and tasks
-> networked nodes and distributed state

```

这张文本图不是排版装饰，而是阅读本册的索引。越往右，通信成本越高，故障模式越复杂，确定性越弱。第一册主要覆盖左侧两层，本册从中间向右推进。

1.4 计算成本的基本单位

分析性能时，不能只看“执行了多少行代码”。更稳定的单位是指令数、周期数、访存字节数、cache miss、TLB miss、分支误预测、同步等待时间、上下文切换、系统调用次数和网络往返延迟。不同单位之间不能随便相加，但它们共同解释程序为什么慢。

例如，一个数组求和循环的算术量很小，主要成本是把数据从内存送到核心。一个矩阵乘如果分块得当，会重复使用缓存中的数据，瓶颈可能转向浮点执行端口。一个共享队列如果每次 push 都抢同一把锁，瓶颈不是算术，也不是内存带宽，而是临界区串行化和 cache line 所有权迁移。

1.5 把成本转成证据

第二册不鼓励凭经验猜瓶颈。每个成本单位都应该有可观察证据。依赖链问题可以用汇编、优化报告、IPC 和不同累加器版本比较；分支问题可以用输入分布、branch-misses 和 branchless 版本比较；cache 问题可以用工作集大小、访问步长和 cache 事件比较；TLB 问题可以用页数、大页和 dTLB 事件比较；锁问题可以用等待时间、futex、context switches 和吞吐曲线比较；NUMA 问题可以用 first-touch、绑定策略和远端访问计数比较；分布式问题可以用 trace、队列长度、重试次数和超时分布比较。

证据的作用不是一次性证明真理，而是排除错误方向。若一个归约程序在线程数增加后不再加速，可能原因很多：内存带宽到顶、false sharing、锁竞争、线程迁移、任务粒度太小、NUMA 放置错误、CPU 频率下降或 benchmark 把初始化混入热路径。没有证据时，这些解释都像对；有了证据，才能逐步缩小范围。

1.6 延迟、吞吐和并发度

系统分析经常混淆三个词：延迟、吞吐和并发度。延迟是单个操作从开始到结束的时间，吞吐是单位时间完成多少操作，并发度是同时在系统中飞行的操作数量。一个内存 load 的延迟可能很高，但如果核心能同时挂起多个 miss，整体吞吐仍然可以接受；一个网络请求延迟很高，但如果客户端保持足够多的 in-flight 请求，吞吐可以接近带宽上限。

这三个量之间有一个朴素但非常有用的关系：要填满一个延迟很高的管道，需要足够并发度。磁盘、网络、内存和 CPU 后端都可以用这个心智模型理解。区别在于，CPU 的并发度来自乱序窗口、load buffer、SIMD lane 和多核心；网络系统的并发度来自连接数、请求队列和异步 I/O；分布式计算的并发度来自分片、任务和流水线。

心智模型

延迟回答“一个操作要等多久”，吞吐回答“系统每秒能做多少”，并发度回答“为了达到吞吐，需要同时容纳多少未完成工作”。

1.7 Amdahl 和 Gustafson

多核性能讨论离不开可并行比例。Amdahl 定律说明，如果程序有一部分必须串行执行，那么线程数增加到一定程度后，加速会被串行部分限制。Gustafson 定律从另一个角度看问题：如果随着核心数增加而扩大问题规模，可并行部分也随之增加，系统仍然能获得更高总吞吐。

这两个定律不是考试公式，而是工程判断工具。一个服务端请求路径中，如果日志锁、全局分配

器、单分片元数据或单线程事件循环占比很高，增加 worker 不会线性加速。一个离线计算任务如果可以把输入切成更多分片，让每个节点处理独立数据，再做树形合并，扩大规模时就更接近 Gustafson 的场景。

1.8 观察系统的三张图

学习计算系统时，建议每遇到一个程序都画三张图。第一张是数据流图：输入从哪里来，经过哪些结构，输出到哪里。第二张是执行图：哪些线程、任务或节点执行哪些阶段，哪里并行，哪里等待。第三张是成本图：每个阶段主要消耗 CPU、内存、I/O、锁等待还是网络。

这三张图可以避免把问题混成一团。一个队列积压，可能是消费者计算慢，也可能是生产者太快，也可能是下游 I/O 阻塞，也可能是队列锁竞争。只有先画清数据和执行边界，测量才知道该放在哪里。

1.9 正确性先于性能

并发和分布式计算最容易出现的错误，是把速度放在正确性之前。数据竞争可能让程序偶尔错；错误的内存序可能只在某些 CPU 上触发；无锁结构如果没有处理对象回收，可能在压力测试里才出现悬垂访问；分布式系统如果没有处理超时和重试，可能在网络抖动时重复提交请求。

本册每次讨论优化，都要先给出语义边界。共享变量由谁写，谁读，什么时候发布，什么时候回收。任务是否可以重排，结果是否依赖顺序。消息是否幂等，故障后是否可以重试。只有这些问题说清楚，性能数字才有意义。

1.10 从单机到分布式的一条线

很多读者会把并发和分布式分成两门课：前者讲线程，后者讲网络。第二册刻意把它们放在一条线上，是因为它们面对的是同一种基本矛盾：多个执行实体共享某些状态，但共享状态越多，协调成本越高。在单机上，协调成本表现为锁、原子、cache line 迁移、调度和内存序；在集群中，协调成本表现为 RPC、复制、共识、重试、超时和数据迁移。

因此，一个好的单机设计原则常常能迁移到分布式系统。线程本地计数再合并，对应 map-side combine；分片锁对应按 key 分区；树形归约对应分层聚合；有界队列和背压对应限流和排队控制；线性化点对应协议中的提交点；对象生命周期对应消息幂等和状态恢复。反过来，分布式系统也会提醒单机程序：只要协调成本足够高，就必须减少共享、批量通信、明确失败语义。

1.11 本册贯穿实验

第二册的实验项目叫 Compute Systems Labs。第一阶段实现顺序归约、并行归约、带锁计数器、原子计数器和有界队列。后续章节会围绕它继续扩展：加入线程池、任务图、工作窃取、无锁队列、并行扫描、I/O 背压和分布式模拟。

实验一：运行 Compute Systems Labs。记录 CPU 型号、核心数、线程数、缓存层级、系统版本、编译器版本和测试命令。不要只写测试通过，还要解释每个模块分别验证了计算系统的哪一类问题。

第 2 章

核心流水线、乱序执行与分支预测

现代 CPU 核心不是按源码顺序一条一条机械执行。为了提高吞吐，核心会把指令拆成微操作，放入队列，分析依赖，尽量让互不依赖的操作并行执行。理解这一层，是理解单线程性能、SIMD 前的标量优化和并发路径的基础。

2.1 流水线不是免费并行

流水线把取指、译码、执行、访存和提交分成阶段。理想情况下，每个周期都有新指令进入，每个阶段都在工作。但真实程序有数据依赖、控制依赖和资源冲突。后一条指令需要前一条指令结果时，执行端口即使空闲也不能使用；分支目标未知时，前端可能取错指令；访存 miss 时，后端会等待数据。

流水线的关键不是“越深越快”，而是吞吐和延迟的权衡。深流水线可以提高频率，却放大分支预测失败的代价。并发程序中的短临界区也会受流水线影响：一个看似只有几条指令的锁操作，背后可能包含原子读改写、cache line 独占获取和内存序约束。

2.2 前端：取指、译码和微操作

核心前端负责把指令流喂给后端。它要从 instruction cache 取指，预测下一条指令地址，译码复杂指令，把它们转成内部微操作，并放入队列。前端跟不上时，后端即使有空闲执行端口也没有工作可做。大函数、过多分支、间接跳转、指令 cache miss 和译码压力都可能让前端成为瓶颈。

x86-64 指令长度可变，译码比定长指令集复杂。现代核心通常会使用 micro-op cache 或类似结构缓存译码结果，让热循环不用每次重新译码。这个机制解释了为什么热循环体大小、函数内联和代码布局会影响性能。内联可以减少调用开销，但过度内联会膨胀代码，伤害 instruction cache 和前端供给。

第一册已经教读者读汇编。第二册读汇编时要多看一个维度：这段机器码对前端是否友好。短小热循环、稳定分支、连续代码布局和可预测调用目标，通常更容易让前端持续供给后端。反过来，大量模板实例化、过度内联、异常路径混在热路径、虚调用目标多变、解释器式大 switch，都可能让前端成本超过源码直觉。

深入理解

前端瓶颈的一个典型现象是：数据已经在缓存中，算术也不复杂，但 IPC 仍然低，并且改变数据布局帮助不大。这时应检查代码体积、分支目标、间接跳转和译码压力，而不是继续只改数组排列。

2.3 后端：执行端口和调度

后端接收微操作后，要等待操作数准备好，并把它们发射到合适执行端口。整数 ALU、浮点乘加、加载、存储、分支和地址生成都有各自资源。一个循环如果每轮需要太多 load，可能受加载端口限制；如果每轮地址计算复杂，可能受地址生成单元限制；如果依赖链很长，端口空着也不能发射。

因此分析单核性能时，要区分“端口吞吐不够”和“依赖导致发不出去”。多累加器能改善后者；SIMD 能提高每条指令处理的数据量；简化地址表达式和连续访问能减轻加载和地址生成压力。不同 CPU 微架构端口配置不同，严肃优化必须结合具体机器。

后端不是一个抽象黑箱。一个 load 通常需要地址生成、TLB 查询、缓存访问和可能的 miss 处理；一个 store 通常拆成地址和数据路径，并进入 store buffer；一个浮点 FMA 会占用特定向量执行资源；一个整数分支会占用分支执行资源。循环的性能取决于这些微操作怎样分布到端口上，也取决于它们之间是否互相等待。

这解释了为什么源码层面的“操作次数”常常误导。两个循环都执行同样数量的加法，一个可能因为单累加器依赖链而慢，另一个可能因为多累加器让后端并行发射而快。两个循环都做同样数

量的 load，一个可能连续访问被预取器支持，另一个可能指针追踪导致每次 load 等上一次 load 的地址。

2.4 乱序执行的边界

乱序执行允许核心在不改变单线程可观察结果的前提下，提前执行已经准备好的指令。寄存器重命名消除了很多假依赖，保留站和重排序缓冲区让多个指令同时等待资源。这样做可以隐藏一部分延迟，例如在一次乘法等待时执行另一个独立加法。

但是乱序执行不能越过所有边界。真实数据依赖不能消除，cache miss 太多时可并行等待的 miss 数量有限，内存屏障会限制重排，原子操作会带来更强的顺序约束。写高性能代码时，增加独立累加器、减少循环携带依赖、让数据访问可预测，都是在帮助乱序核心找到可调度空间。

2.5 重排序缓冲区和执行窗口

乱序执行不是无限的。核心只能观察一个有限窗口内的指令。重排序缓冲区保存尚未提交的指令，load buffer 保存未完成加载，store buffer 保存尚未对外可见的存储。如果窗口被长延迟 load 占住，后续可执行工作又不够，核心会停顿。

链表遍历就是典型例子。每次读取下一个指针之前，必须等当前节点加载完成。乱序核心很难提前知道下一步地址，所以内存级并行度很低。相比之下，遍历多个独立数组或同时处理多条链，可以让多个 load 同时飞行，更容易隐藏延迟。

执行窗口可以被理解为硬件能同时“看见”的未来。窗口越大，越有机会在一个长延迟操作等待时找到其他独立工作。但如果代码中充满循环携带依赖，窗口再大也找不到可提前执行的操作；如果 cache miss 太多，load buffer 被占满，后续加载也发不出去；如果分支预测失败，窗口里的错误路径工作会被丢弃。

软件能做的事情，是把独立工作显式放进窗口。多累加器、分块处理、批量处理多个请求、一次处理多个链表游标、把检查从热循环中外提，都属于给乱序核心提供更多可调度空间。相反，频繁调用不可内联函数、隐藏别名、每步都依赖共享原子结果，会压缩这个空间。

2.6 Load、Store 和内存消歧

内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store。如果地址关系不明确，硬件会做内存消歧预测。预测错误时，需要回滚并重新执行相关操作。高性能循环中，减少指针别名、使用清晰的数组边界、避免复杂交叉读写，可以帮助硬件和编译器更大胆地调度。

store buffer 也影响并发语义。普通 store 可以先进入 store buffer，稍后再对其他核心可见。内存屏障、原子操作和锁释放会约束这种行为。C++ 内存模型的 acquire/release 最终要落到这类硬件机制上。

Load/store queue 还是很多微妙性能问题的来源。若一个循环中既写数组又读另一个可能别名的数组，编译器和硬件都必须保守处理读写顺序。若 store 地址很晚才算出来，后面的 load 就难以确认是否可以提前。若 store buffer 堆积，后续 store 可能被阻塞，锁释放和原子操作也可能放大这个问题。

并发程序中，store buffer 让“本线程已经执行了写”不等于“其他线程已经能按你想象的顺序看到写”。这不是说硬件不可靠，而是说可见性需要同步语义。后面讲 release/acquire 时，会把这条硬件直觉变成 C++ 层面的 happens-before 关系。

2.7 分支预测和控制流

分支预测让核心在条件结果出来之前先猜路径。预测正确时，流水线保持饱满；预测失败时，错误路径上的工作要被丢弃。随机分支、数据相关分支和复杂间接跳转都会破坏预测。很多高性能循环会把条件变成掩码、查表或分段处理，就是为了减少不可预测控制流。

并发程序也受分支预测影响。锁的 fast path 通常期望“不竞争”，如果竞争比例上升，分支行为和 cache 行为都会改变。无锁结构的 CAS 循环在低竞争时很短，在高竞争时会反复失败，控制流和内存系统同时恶化。

深入理解

判断一段代码是不是受分支预测影响，可以比较不同输入分布下的 cycles、branches 和 branch-misses。如果随机数据显著慢于有序数据，而内存访问量没有明显变化，分支预测通常就是主要嫌疑。

2.8 SMT 和共享核心资源

SMT 让一个物理核心暴露多个逻辑 CPU。它可以在一个线程等待内存或前端供给不足时，让另一个线程使用空闲资源。但 SMT 不是免费翻倍。两个逻辑线程会共享前端、执行端口、缓存、TLB 和乱序资源的一部分。如果两个线程都在做同样高强度向量计算，它们可能互相抢执行资源；如果一个线程经常等内存，另一个线程可能填补空档。

因此 benchmark 中不能只写“8 个 CPU”。要区分物理核心和 SMT 线程。对于 CPU-bound 数值内核，先扩展到物理核心往往比立刻使用所有逻辑 CPU 更稳定；对于延迟隐藏明显的工作负载，SMT 可能提高吞吐。结论必须来自本机测量，而不是固定经验。

2.9 微架构和 ISA 的边界

第一册强调 ISA 和 ABI：同一条 `add` 在 x86-64 语义上做什么，函数参数怎样传递。第二册强调微架构：同一个 ISA 指令在不同 CPU 上可能拆成不同微操作，使用不同端口，拥有不同延迟和吞吐。ISA 给出程序可见语义，微架构决定实现成本。

这条边界非常重要。写教材时不能把某个 CPU 的端口配置说成 x86-64 的永久性质；写性能报告时也不能只说“x86 上这个指令很快”。更准确的说法是：在某个 CPU 型号、某个编译选项、某个输入分布下，观察到某个指令序列的瓶颈更像前端、端口、依赖、内存或分支。

2.10 典型性能症状

前端瓶颈常表现为 IPC 低、branch miss 或 instruction cache 相关事件高，代码布局和热循环大小值得检查。后端执行瓶颈常表现为数据在缓存中、分支也稳定，但 IPC 仍受端口或依赖限制，适合检查 SIMD、多累加器和指令混合。内存瓶颈常表现为 cache miss、TLB miss 或带宽接近平台上限。同步瓶颈则可能表现为 IPC 低、cache line 迁移、context switch、futex 等待或原子热点。

这些症状不会自动给答案，但能排除方向。不要在内存带宽瓶颈上执着减少几条整数指令，也不要锁竞争瓶颈上只调整循环展开。性能工程的价值在于把“慢”拆成可以证伪的假设。

2.11 从硬件回到代码

核心流水线给软件三个直接启发。第一，减少长依赖链，例如把单个累加器拆成多个局部累加器。第二，减少不可预测分支，让热路径尽量直。第三，减少会强制顺序的操作，例如不必要的原子和屏障。后续章节讨论 SIMD、锁、无锁队列和并行归约时，都会回到这些原则。

实验：写两个求和版本。第一个使用单累加器，第二个使用四个独立累加器。用 `perfstat` 比较 cycles、instructions 和 IPC。再构造一个带随机分支的版本，观察 branch-misses 对时间的影响。

第 3 章

缓存、TLB 与页级性能

CPU 核心很快，主内存相对很慢。缓存层级的存在，是为了让核心大多数时候从更近、更小、更快的存储中取数据。虚拟内存的存在，是为了让进程拥有独立地址空间、权限保护和按需映射。第一册已经讲过进程地址空间、映射和缺页的基本概念；本章不重复这些基础，而是继续追问：地址翻译、缓存映射、写共享、page walk、首次触页和 NUMA 放置怎样限制多核程序的吞吐。

3.1 缓存层级与 cache line

缓存不是按单个字节移动数据，而是按 cache line 移动。常见 cache line 大小是 64 字节。顺序访问数组时，一次 miss 可以带来后续多个元素；随机访问链表时，每个节点可能引入一次新的 miss。这个差异解释了为什么连续数组常常比指针结构更快。

缓存有容量、相联度和替换策略。容量不足会产生容量 miss，相联度不足会产生冲突 miss，多核心共享写会产生一致性流量。一个算法理论复杂度更低，不代表真实更快；如果它把连续访问变成随机访问，可能输给复杂度略高但局部性更好的算法。

从第二册的角度看，cache line 还是多核协作的最小一致性单元。两个线程哪怕写的是两个不同字段，只要字段落在同一条 cache line 上，硬件就必须在核心之间移动同一个一致性单元。很多并发程序的扩展性问题，不是算法复杂度问题，而是写入布局问题。后面讲锁、原子和无锁队列时，都要回到这个事实。

```
one cache line, 64 bytes in a common x86 machine

| counter[0] | counter[1] | counter[2] | counter[3] | ...
    T0         T1         T2         T3

looks independent in source code
shares one coherence unit in hardware
```

这类图要让读者形成一个直觉：源码里的变量边界，不等于硬件里的传输边界。硬件搬的是 cache line，TLB 记的是页，操作系统调度的是线程，分布式系统发送的是消息。边界错位，是系统性能问题最常见的来源之一。

3.2 缓存映射和冲突

缓存通常按 set associative 组织。一个地址会映射到某个 set，同一个 set 里只有有限个 way。如果多个热点地址恰好映射到同一个 set，而数量超过相联度，就会频繁互相驱逐，即使总工作集小于缓存容量也会慢。这类冲突 miss 很隐蔽，因为从容量角度看“不应该放不下”。

矩阵步长、数组对齐和结构体大小都可能触发冲突。若多个数组按相同大步长访问，并且基地址关系不佳，它们可能持续争用同一组 cache set。改变 padding、改变块大小或调整访问顺序，有时能显著改善性能。

3.3 Inclusive、Exclusive 和共享缓存

不同处理器的缓存层级策略不同。有的层级倾向 inclusive，也就是上层 cache 的内容也在下层保留；有的倾向 exclusive，尽量让不同层级存放不同内容；还有很多混合策略。软件通常不需要依赖某个具体策略，但要知道共享 LLC 不是无限资源。多个线程在同一 socket 上跑不同任务，可能互相驱逐 LLC。

这对并行任务调度有直接影响。把共享数据的任务放在共享缓存附近，可能复用 LLC；把互相干扰的大流式任务放在同一 socket，可能争抢带宽和 LLC。操作系统调度器会做一般性决策，但高

性能程序常需要自己记录拓扑并做亲和性实验。

3.4 写路径和 false sharing

读缓存可以共享，写缓存必须取得 cache line 的独占所有权。两个线程写不同变量，如果这些变量落在同一条 cache line 上，硬件仍然会把它当成同一块一致性单元反复迁移，这就是 false sharing。它不是锁竞争，却会表现出类似的扩展性下降。

解决 false sharing 的方法包括按线程分配独立槽位、使用对齐和填充、批量合并结果、减少共享写频率。并行归约通常先让每个线程写自己的 partial sum，最后再合并，正是为了避免每次加法都写同一个共享变量。

Listing 3.1 用对齐槽位减少 false sharing

```
1 struct alignas(64) LocalCounter {
2     std::int64_t value = 0;
3 };
4
5 std::vector<LocalCounter> counters(worker_count);
6 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
7     counters[static_cast<std::size_t>(worker)].value += 1;
8 }
```

这段代码不是说所有结构体都要强行 64 字节对齐，而是表达一个设计原则：如果多个线程会高频写不同槽位，这些槽位不要共享同一条 cache line。否则程序表面没有共享变量，硬件层面仍然在共享一致性单元。

false sharing 的危险在于它不像数据竞争那样直接破坏结果。程序可能完全正确，测试也全部通过，只是在核心数增加时吞吐突然停止增长。排查时要看写入频率、字段布局、线程到槽位的映射和 cache line 迁移事件。若没有 `perfctr` 或类似工具，也可以用对齐填充做对照实验：只改变布局，不改变算法。如果对齐版本显著更快，就说明共享写路径值得重点检查。

3.5 读共享和写共享

共享数据不总是坏事。只读共享表、常量配置、不可变索引和代码段可以被多个核心同时缓存，通常扩展性很好。真正昂贵的是写共享，尤其是多个核心高频写同一条 cache line。读多写少的数据可以使用读写锁、RCU、版本化快照或 copy-on-write，目的都是让读路径尽量不进入写共享热路径。

写共享也分层次。最差情况是每次操作都写同一个全局变量，例如所有线程对一个原子计数器做 `fetch_add`。较好的情况是每个线程写本地槽位，周期性合并。更好的情况是让写入天然按数据分片分散，例如按 key 分片的哈希表或按 NUMA 节点分组的内存池。布局、同步和算法在这里是同一个问题的三个面。

3.6 TLB 和页表

虚拟地址访问物理内存前需要地址翻译。TLB 缓存虚拟页到物理页的映射。工作集页数太多或访问模式太随机时，TLB miss 会增加，核心需要走页表。大页可以减少页数，提高 TLB 覆盖范围，但也会影响内存分配、NUMA 放置和碎片。

理解 TLB 对数据结构设计很重要。一个跨很多小对象的图遍历，不只会造成 cache miss，还可能造成 TLB miss。紧凑存储、池化分配和按块遍历，可以同时改善 cache 和 TLB 行为。

TLB 的特殊之处在于，它限制的是“能高效覆盖多少虚拟页”。一个程序的工作集如果只看字节数不大，但分散在大量页面上，仍然可能受到 TLB miss 影响。链表、树、哈希表桶、对象池和图邻接结构都可能出现这个问题。把对象压缩到连续数组里，不只是减少 cache miss，也是在减少活跃页数。

3.7 Page walk 的成本

TLB miss 后，硬件需要按页表层级查找映射。x86-64 常见四级或五级页表，每一级页表项本身也在内存里，虽然页表项可能被缓存，但最坏情况下一个地址翻译会触发多次内存访问。若程序随机访问大量页面，真正拖慢它的不只是数据 load，还有地址翻译。

大页把页大小从 4 KiB 扩大到 2 MiB 或更大，显著增加 TLB 覆盖范围。它适合大数组、数据库缓存和数值计算工作集，但不适合所有场景。大页可能增加内存碎片，NUMA 放置也要更谨慎。使用大页前应先使用计数器确认 TLB 确实是瓶颈。

Page walk 还会和缓存层级交织。页表项本身也要从内存层级读取，也可能命中或错过缓存。一个随机访问大数组的程序，表面上每次只读一个元素，实际上可能同时触发数据 cache miss 和页表访问。若工作集跨越大量页面，TLB miss 的成本会叠加在数据访问成本上。优化时不能只看“每个元素读几个字节”，还要看“每个时间窗口内活跃多少页”。

3.8 缺页和内存分配

分配内存不等于物理页已经准备好。操作系统通常按需分配页面，第一次触碰页面时可能发生 page fault。benchmark 如果把第一次触碰计入核心循环，会把页错误成本混入算法时间。严谨实验通常要预热、固定输入规模，并记录 page-faults。

缺页也分 minor fault 和 major fault。minor fault 不需要从磁盘读数据，可能只是建立页表映射；major fault 需要从存储设备读取，成本高得多。高性能计算通常要避免在热路径出现 major fault，也要避免把大量 minor fault 误判为算法本身慢。

第一册已经要求 benchmark 避免把初始化混进热路径。第二册要进一步要求初始化方式和计算方式一致。NUMA 机器上，如果主线程单独触碰全部页面，再让多个 socket 上的 worker 计算，页面可能集中在主线程所在节点，后续计算变成远端内存访问。正确实验通常让每个 worker 初始化自己之后要处理的分块，这样 page fault、物理页分配和计算拓扑保持一致。

3.9 从虚拟内存到 mmap

mmap 可以把文件或匿名内存映射到进程地址空间。它让文件 I/O 看起来像内存访问，但不意味着没有 I/O 成本。第一次访问未驻留页面时仍然会缺页，顺序扫描大文件时内核预读可能有效，随机访问时可能产生大量 page fault。

数据库、搜索引擎和日志系统常利用 mmap 或 page cache，但必须理解内核何时回收页面、何时写回脏页、何时产生 I/O 抖动。系统工程里，虚拟内存既是便利抽象，也是性能变量。

3.10 Linux 源码阅读入口

第一册的 Linux 源码阅读只要求读者能从用户态现象找到入口。第二册要读出数据结构和性能路径。围绕本章，可以固定一个 Linux 版本，沿下面几条路径做窄范围阅读：页错误入口可从体系结构相关的 fault handler 进入，再走到通用内存管理逻辑；VMA 管理可关注 mm/mmap.c 和相关 maple tree 结构；页表和 fault 处理可关注 mm/memory.c；NUMA 策略可关注 mm/mempolicy.c；page cache 和文件映射可从 mm/filemap.c 开始。

源码阅读报告不要写成“Linux 使用某某机制”这么宽泛。合格报告要写清内核版本、阅读路径、关键结构、用户态实验和结论边界。例如研究首次触页，可以写一个大数组实验，记录 page-faults，再阅读 fault 路径中如何建立匿名页映射。若当前设备权限不足，仍然可以保留源码阅读，但必须承认它不是当前运行内核的直接证据。

3.11 本章的设计原则

缓存和 TLB 优化可以压缩成四条原则。第一，让热路径连续访问，减少随机指针跳转。第二，让共享写分散，避免多个线程争用同一条 cache line。第三，让工作集按块复用，避免超出缓存后仍反复访问。第四，让页面放置和线程执行位置一致，避免首次触页造成远端访问。这四条原则会贯穿后面的数据布局、并行归约、线程池、工作窃取和分布式 shuffle。

常见误区

不要把 `std::vector` 的连续性理解成所有成本都消失了。连续布局改善 cache line 和预取，但如果容量超过缓存，仍然会受内存带宽限制；如果首次访问大量新页，仍然会看到缺页成本。

实验：比较行优先访问矩阵和列优先访问矩阵。记录 cache-misses、dTLB 相关事件和时间。再把矩阵规模从 L1、L2、L3 可容纳范围逐步扩大，观察拐点。

第 4 章

NUMA、互连与缓存一致性

多核心 CPU 不是把多个核心简单放在一起。核心之间通过片上互连通信，共享某些缓存层级，连接内存控制器，并用缓存一致性协议维护共享内存语义。多 socket 机器还会引入 NUMA：不同核心访问不同内存节点的成本不同。

4.1 共享内存的硬件代价

在 C++ 程序里，多个线程似乎可以直接读写同一地址。但硬件上，写一个共享 cache line 需要让其他核心的副本失效或更新。一个原子自增不是普通加法，它需要获得 cache line 独占权，并按内存序约束发布结果。争用越激烈，cache line 在核心之间来回迁移越频繁。

这解释了为什么全局计数器扩展性差。四个线程分别做本地计数，最后合并，通常比四个线程同时 `fetch_add` 同一个原子变量快。前者减少共享写，后者让每次更新都进入一致性协议热路径。

4.2 MESI 的直觉

很多一致性协议都可以用 MESI 直觉理解：Modified 表示本核心有被修改且尚未写回的独占副本，Exclusive 表示本核心有未修改的独占副本，Shared 表示多个核心可以读，Invalid 表示副本无效。当一个核心要写 Shared 行时，需要让其他核心副本失效；当其他核心随后要读，又要重新获取数据。

软件看不到这些状态名，但能看到它们的后果。一个只读共享表可以被多个核心同时缓存，扩展性很好；一个频繁更新的共享 head 指针会让 cache line 在核心之间跳来跳去；一个包含多个线程本地字段的结构体如果没有 padding，也会因为 false sharing 触发同样的迁移。

真实处理器可能使用 MESIF、MOESI 或其他变体，但教学时先抓住一个核心事实：读共享容易扩展，写共享需要取得所有权。Modified 或 Owned 这类状态的具体名称不重要，重要的是写入会触发状态迁移，状态迁移会占用互连和缓存层级资源。高性能并发设计的第一反应，不应该是“换一条更快的原子指令”，而应该是“能不能减少共享写”。

```
read mostly table:
  core0 Shared
  core1 Shared
  core2 Shared

hot atomic counter:
  core0 Modified -> core1 Modified -> core2 Modified -> core0 Modified
```

上面的文本图表达两种完全不同的共享。前者多个核心一起读，硬件可以复制；后者多个核心轮流写，所有权不断迁移。很多扩展性问题就藏在这条差异里。

4.3 一致性和内存模型的分工

缓存一致性解决的是“同一内存位置的多个副本如何保持一致”，内存模型解决的是“不同内存操作之间能被哪些线程以什么顺序观察”。硬件一致性不等于程序自动无数据竞争。没有同步的普通读写仍然可能在 C++ 层面形成未定义行为。

一致性协议通常围绕状态机工作，例如 Modified、Exclusive、Shared、Invalid 这类状态。软件不需要手写这些状态，但必须理解状态迁移的成本：共享读便宜，共享写贵，频繁写同一行最贵。

C++ 内存模型和硬件一致性的边界也要分清。硬件一致性通常保证同一 cache line 或同一地址的副本最终按协议协调，但 C++ 程序如果对普通变量并发读写而没有同步，语言层面就是数据竞争。语言规则先决定程序是否有定义，硬件规则再决定有定义的同步怎样落地。不要用“我的 CPU

有缓存一致性”来为未同步普通变量辩护。

4.4 Store buffer 和可见性

核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还受缓存一致性和内存序约束影响。锁释放、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这就是为什么“我在写 flag 前已经写了 data”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象需要用 release/acquire、mutex 或其他同步机制建立 happens-before。

Store buffer 还会影响锁的性能。释放锁时，持锁线程在临界区内的写入必须以满足同步语义的方式对后续拿锁线程可见。获取锁时，线程可能需要等待锁所在 cache line 的最新状态。低竞争时这条路径很短；高竞争时，锁变量所在 cache line 会成为所有核心争夺的热点。锁的成本不是一个固定常数，而是竞争、拓扑和写入路径共同决定的结果。

4.5 NUMA 放置

NUMA 机器上，内存离某些核心更近。线程在节点 A 上运行，却频繁访问节点 B 分配的内存，会产生远端访问。Linux 的 first-touch 策略让首次触碰页面的线程影响页面放置，因此并行初始化和线程绑定会影响后续计算性能。

NUMA 优化的基本原则是让计算靠近数据。大数据组按块分给线程时，初始化也应按相同块并行完成。跨节点共享队列和全局分配器要谨慎，因为它们可能把远端访问和同步争用叠加在一起。

NUMA 不只出现在多 socket 服务器上。有些桌面和移动平台也有非均匀的缓存、内存或芯片 let 拓扑，只是操作系统暴露方式不同。教材里使用 NUMA 这个词，是为了让读者形成拓扑意识：核心不是一个无差别集合，内存也不是一个无差别池。线程、数据和 I/O 设备之间的位置关系会影响成本。

4.6 First-touch 和并行初始化

Linux 常见 first-touch 策略意味着页面通常分配到第一次写入它的线程所在 NUMA 节点。如果主线程单线程初始化一个大数组，然后工作线程分布在多个节点上处理这个数组，很多工作线程可能访问远端内存。更好的方式是让每个工作线程初始化自己后续要处理的分块。

这条规则在 benchmark 中尤其重要。若初始化方式和计算方式不一致，测到的可能是内存放置错误，而不是算法本身。实验报告必须记录初始化策略、线程绑定和 NUMA 策略。

4.7 互连不是无限带宽

核心、缓存、内存控制器和 I/O 设备之间的互连带宽有限。一个程序即使单线程局部性很好，多线程扩展后也可能撞上内存带宽或互连带宽上限。此时继续增加线程，只会让每个线程分到更少带宽，吞吐不再提高。

互连瓶颈常常表现为“每个线程单独跑都快，一起跑就都慢”。原因可能是多个核心争用同一内存控制器，多个 socket 之间远端访问过多，或者共享写导致 cache line 在互连上来回移动。排查时要把线程数、绑定位置、数据放置和访问模式作为实验矩阵，而不是只改算法代码。

4.8 拓扑感知设计

拓扑感知不是过早优化，而是避免把硬件当作平面资源池。线程池可以按 NUMA 节点分组，内存池可以按节点分配，队列可以按生产者或消费者分片，归约可以先在 socket 内合并再跨 socket 合并。这样的层次化设计和分布式系统中的本地聚合很像：先减少近处流量，再把少量结果发到远处。

4.9 Linux 观察入口

在 Linux 上，拓扑可以从 `lscpu`、`/sys/devices/system/cpu/`、`/sys/devices/system/node/` 和 `numactl--hardware` 开始观察。线程绑定可以用 `taskset`、`numactl--physcpubind` 或程序内亲和性 API。内存策略可以用 `numactl--membind`、`numactl--interleave` 和 `first-`

touch 实验观察。若权限允许，`perf` 可以帮助识别 cache-to-cache 流量，`perfstat` 的 cache、LLC、context-switch 和迁核指标也能提供间接证据。

源码阅读可以从调度拓扑、NUMA 策略和内存分配路径切入。不要试图一次看完整调度器或完整内存管理。更好的问题是：线程迁移时哪些状态会变化，first-touch 影响页面放置的路径在哪里，NUMA 策略怎样被记录和查询，某个 benchmark 的现象能不能从这些路径得到合理解释。

4.10 层次化归约案例

并行归约是理解拓扑的好例子。最差设计是所有线程对同一个原子变量累加。改进设计是每个线程写自己的 partial，最后一个线程合并。进一步的拓扑感知设计是每个核心或每个 worker 先本地累加，每个 NUMA 节点内再合并，最后跨节点合并。这样做减少共享写，也减少远端通信。

这个结构和分布式计算中的树形聚合完全一致。不同只是成本单位变化：单机里远处是另一个 NUMA 节点，分布式里远处是另一台机器。理解这条相似性，可以帮助读者把第二册前后内容串起来。

实验：在支持 NUMA 的机器上，用 `numactl--hardware` 查看节点。写一个大数组初始化和求和程序，比较单线程初始化、多线程初始化、线程绑定和未绑定情况下的带宽。如果没有 NUMA 机器，也要用 `lscpu` 记录拓扑并解释限制。

第零部分

单机高性能计算：从测量到数据流

第 5 章

可信测量、Roofline 与性能计数器

性能工程必须先建立测量纪律。第一册已经讲过输入规模、环境、编译选项、计时方式、热路径隔离、正确性校验和基础 `perfstat`。本章不重复这些基本规则，而是讨论第二册真正需要的进阶测量：如何把多核程序分解成计算、内存、分支、TLB、同步、调度和 I/O 假设；如何用 Roofline 判断上限；如何用性能计数器和对照实验排除错误解释。

5.1 测量对象

测量前先定义对象。是测单次操作延迟，还是测批量吞吐；是包含内存分配，还是只测热循环；是冷缓存，还是热缓存；是单线程，还是多线程；是平均值，还是尾延迟；是关心总吞吐，还是关心每个 worker 是否均衡。不同对象需要不同方法，不能混用结论。

常见错误包括：把初始化时间算进内核，把编译器优化掉的循环当作结果，把首次缺页当作算法成本，把输出打印放进计时区，把不同 CPU 频率状态下的结果直接比较。

第二册还要特别避免一种错误：只测总时间，不看扩展曲线。多线程程序的核心问题不是“某一次运行快不快”，而是从 1 个线程到多个线程时，吞吐如何变化。一个程序在 2 个线程时加速，在 8 个线程时停滞，在 16 个线程时变慢，这条曲线本身就是证据。它提示瓶颈可能从单核执行转移到了内存带宽、锁竞争、调度、NUMA 或 I/O。

5.2 计时器和统计方法

计时器应使用单调时钟，避免系统时间调整影响。短任务要重复执行足够多次，并防止编译器消除结果。报告结果时不要只给一次运行时间，应至少给出多次运行的最小值、中位数和波动范围。最小值常代表干扰较少的情况，中位数更接近日常情况，尾部异常则提示系统噪声或资源竞争。

预热同样重要。第一次运行可能包含页错误、动态链接、分支预测冷启动、缓存冷启动和 CPU 频率爬升。若目标是冷启动延迟，就应该保留这些成本；若目标是热循环吞吐，就应该把它们排除。测量对象不同，预热策略不同。

多核 benchmark 还要记录线程绑定和运行顺序。若每轮测试先跑单线程再跑多线程，CPU 温度、频率和缓存状态可能系统性变化。更稳妥的做法是随机化版本顺序，保留原始样本，并记录每轮线程数、绑定策略和输入位置。若差异小于系统波动，就不要写强结论。

5.3 Roofline 模型

Roofline 用算术强度连接计算峰值和内存带宽。算术强度是操作数与数据移动字节数的比值。低算术强度程序通常受内存带宽限制，高算术强度程序才可能接近计算峰值。这个模型不是精确预测器，而是帮助判断优化方向。

数组求和每读取一个整数只做一次加法，算术强度低；矩阵乘如果分块得当，每个元素可以被多次复用，算术强度高。若一个内核已经接近内存带宽上限，继续手写更复杂的算术指令可能没有意义；若内核远低于计算上限且数据复用充分，就要检查 SIMD、指令级并行和执行端口。

Roofline 的价值在于先给出“最多可能多快”的粗上限。假设一个内核每处理一个元素读取 8 字节、写入 8 字节，只做少量整数运算，那么它的算术强度很低。若平台可持续内存带宽是 50 GiB/s，那么不管循环写得漂亮，只要每个元素必须从内存读写 16 字节，吞吐上限就被数据移动限制。反过来，若内核通过分块让同一批数据被重复使用，算术强度提高，才有资格讨论 FMA、SIMD 宽度和执行端口。

心智模型

Roofline 不是为了画漂亮图，而是为了阻止错误优化：内存带宽瓶颈不要只改算术指令，计算端口瓶颈不要只改数据结构，同步瓶颈不要只改循环展开。

5.4 从字节数推导瓶颈

Roofline 的关键是先估算字节数。以 `std::int32_t` 数组求和为例，每个元素读取 4 字节，做一次整数加法。如果数组远大于缓存，主要流量来自内存读取。若机器可持续内存带宽是 40 GiB/s，那么理想情况下每秒最多读取约 100 亿个整数。实际还要扣除循环开销、预取效率、TLB 和其他系统干扰。

这个估算能提前判断优化方向。如果实测已经接近带宽上限，就不该期待换一种加法写法获得数量级提升。若实测只有带宽上限的一小部分，就要检查访问是否连续、是否触发 TLB miss、是否有分支、是否被编译器向量化、是否被线程调度干扰。

字节数估算要区分“算法逻辑字节”和“实际流量”。逻辑上，一个原子计数器每次只更新 8 字节；硬件上，它可能让一整条 cache line 在核心之间来回迁移。逻辑上，一个对象只读取一个字段；硬件上，cache line 可能带入多个冷字段。逻辑上，`mmap` 文件像数组；系统上，第一次访问可能触发缺页和 page cache 读取。第二册的实验报告必须说明这些差异，否则字节数估算会过于乐观。

5.5 性能计数器

Linux 上常用 `perfstat` 查看 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches 等指标。更深入时可以用 `perfrecord` 和 `perfreport` 定位热点。不同 CPU 的事件名称不同，实验报告要记录命令和平台。

计数器要组合解释。IPC 低可能因为 cache miss，也可能因为分支失败或同步等待。cache-misses 高不一定说明程序差，流式扫描大数组本来就会不断 miss，但可能充分利用带宽。context-switches 多可能说明线程过多、阻塞频繁或系统有干扰。

进阶测量更重要的是“假设驱动”。如果怀疑前端瓶颈，就比较指令缓存、分支、uops 供给和代码布局；如果怀疑内存带宽，就比较工作集大小、线程数、带宽工具和 LLC miss；如果怀疑 TLB，就改变页大小或访问页数；如果怀疑锁竞争，就看吞吐曲线、等待时间、futex 路径和临界区长度；如果怀疑 NUMA，就改变 first-touch 和绑定策略。不要一次性收集一大堆事件再事后找故事。

5.6 Top-Down 思维

现代性能分析常把 CPU 周期粗分为前端受限、错误预测受限、后端受限和有效退休。不同平台工具名称不同，但思路相同：先判断核心为什么没有持续退休有用指令，再进入更细分类。前端受限时检查取指、译码、代码布局和间接跳转；错误预测受限时检查分支输入分布；后端受限时继续区分执行端口、内存延迟、内存带宽和同步等待；有效退休比例高时，说明核心大部分时间确实在做有用工作，优化要回到算法或工作量本身。

这套思维可以避免把 IPC 低简单等同于“CPU 不行”。同样是 IPC 低，链表随机访问、锁等待、page fault、系统调用阻塞和分支乱跳的解决方向完全不同。报告里应该写“当前证据更支持哪一类受限”，而不是只列一个 IPC 数字。

5.7 同步和调度的测量

多线程程序经常慢在不退休用户态指令的地方。线程可能睡在 futex 上，可能被调度器切走，可能自旋等待 cache line，可能阻塞在 I/O，也可能因为 cgroup 配额用完而被节流。只看 cycles 和 instructions，容易忽略这些等待。

测同步要记录至少三类事实。第一，吞吐随线程数怎样变化；第二，等待在哪里发生，是用户态自旋、内核 futex、条件变量、I/O 还是队列为空；第三，竞争对象是什么，是一把锁、一个原子变量、一条 cache line、一个队列还是一个分片。若能使用 `perfsched`、`perflock`、`perfc2c` 或 eBPF 工具，应记录工具版本和权限；若不能使用，也要用对照实验替代，例如拆分计数器、增加 padding、改变线程绑定和改变队列容量。

5.8 多核扩展曲线

一个高质量性能报告不只给“优化前后快了多少”，还要给扩展曲线。横轴是线程数或 worker 数，纵轴可以是吞吐、耗时、加速比、效率或尾延迟。理想线性加速很少出现，重要的是解释拐点。若 1 到 4 线程接近线性，8 线程开始变慢，可能是共享缓存、内存带宽、NUMA 或同步热点；若从

2 线程开始就没有提升，可能是任务粒度太小、串行部分太大或全局锁太重。

扩展曲线还要配合正确性校验。并行程序在高线程数下更快但偶尔错误，不是优化成功，而是暴露了竞争。每轮性能样本都应关联校验结果，校验失败的样本不能混入性能统计后继续分析速度。

5.9 实验记录模板

一份合格性能记录至少包含：机器型号、核心和 NUMA 拓扑、内存容量、编译器和编译选项、输入规模、数据类型、线程数、绑定策略、运行次数、计时范围、正确性校验、perf 命令和原始输出摘要。缺少这些信息，别人无法复现实验，你自己几周后也很难判断数字是否可靠。

性能结论要写成果因果假设，而不是口号。例如“并行版本在线程数超过 8 后不再加速，perf 显示 LLC miss 和内存带宽接近平台上限，因此瓶颈更像内存带宽，而不是线程创建成本”。这样的结论可以继续被验证或推翻。

本册建议把报告分为五段。第一段写被测问题和正确性 oracle。第二段写机器、系统、编译和绑定环境。第三段写实验矩阵，包括输入规模、线程数、版本、运行轮次和顺序。第四段写原始结果摘要，不只给平均值，也给中位数、最小值和异常。第五段写归因假设和下一步验证。若证据不足，要明确写“当前只能排除某些解释，不能确认最终瓶颈”。

核心理想

测量不是为了得到一个数字，而是为了排除错误解释，缩小瓶颈范围，并验证优化是否真的改变了瓶颈。

实验：对 Compute Systems Labs 的顺序归约和并行归约运行 `perfstat`。记录时间、cycles、instructions、IPC、cache-misses 和 context-switches。解释并行版本是否真的更快，如果没有，瓶颈可能在哪里。

第 6 章

数据布局、局部性与预取

数据布局决定程序怎样使用缓存、TLB 和内存带宽。很多高性能优化并不是改变算法复杂度，而是把数据摆成硬件更容易消费的形状。理解布局，是理解数组、结构体、队列、图和矩阵内核的共同基础。

6.1 AoS 与 SoA

Array of Structures 把一个对象的字段放在一起，适合经常同时访问完整对象的场景。Structure of Arrays 把同类字段连续存放，适合批量处理某个字段的场景。例如粒子模拟中，如果每轮只更新所有粒子的 x 坐标，SoA 更容易形成连续访问和 SIMD；如果每次处理一个粒子的全部属性，AoS 更直观。

选择布局不能只看代码好不好写，要看热路径读写哪些字段、是否需要向量化、是否有 false sharing、是否跨页跳转。工程中常见折中是 AoSoA，把数据按小块组织，既保留局部对象关系，又适合 SIMD 和缓存块。

Listing 6.1 AoS 与 SoA 的访问差异

```
1 struct Particle {
2     float x;
3     float y;
4     float z;
5 };
6
7 struct ParticleSoA {
8     std::vector<float> x;
9     std::vector<float> y;
10    std::vector<float> z;
11 };
```

如果热路径只更新所有粒子的 x ，SoA 版本会连续读取和写入 x 数组；AoS 版本每次还把 y 和 z 附近的数据带入 cache line。若热路径总是同时使用 x 、 y 、 z ，AoS 的空间局部性反而更自然。

布局选择要从访问矩阵出发，而不是从结构体是否“面向对象”出发。可以把字段列成列，把热路径列成行，在每个格子里标出读、写、只读配置、冷路径调试。若一个热路径只读少数字段，SoA 或冷热分离更有优势；若多个字段总是一起使用，AoS 或 AoSoA 更清晰；若需要 SIMD，字段连续性和对齐会更重要；若需要并发写，线程到字段和槽位的映射必须避免 false sharing。

```
hot path: update position
  reads:  x, y, z, vx, vy, vz
  writes: x, y, z

hot path: compute bounding box
  reads:  x, y, z
  writes: thread local min and max

cold path: debug print
  reads:  name, id, material, history
```

这张访问矩阵会自然推出布局：调试字段不应混在高频数值字段里；只在统计阶段使用的字段

不应污染每帧更新；线程本地输出应分开存放，最后合并。

6.2 预取

硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描通常能被预取器提前加载，随机指针追踪则很难。软件预取可以在少数场景有效，但如果预取太早会挤出有用缓存，太晚则来不及隐藏延迟。

最好的预取优化通常不是手写预取指令，而是改变访问模式：把随机访问排序，按块处理，压缩节点，减少指针跳转，让硬件预取器能看懂程序。

软件预取只有在地址能提前算出、未来确实会访问、当前有足够独立工作、预取距离合适时才可能有效。指针追踪中，如果下一步地址必须等当前节点加载完成，预取很难发挥作用；批量处理多个游标时，才有机会提前为另一个游标发起预取。预取错误还可能污染缓存，把真正热的数据挤出去。

因此本书把预取看成最后一步，而不是第一步。先让数据连续，先分块，先减少对象分散，先测出内存延迟或带宽瓶颈，再考虑预取指令。很多工程代码的性能来自“让硬件预取器自然工作”，而不是手写大量 prefetch。

6.3 冷热数据分离

对象里不是所有字段都同样热。把热字段和冷字段混在一起，会让缓存带入很少使用的数据。冷热分离把热路径频繁访问的字段放在紧凑结构中，把调试信息、统计字段、名字、配置等冷数据放到旁边结构或间接存储。数据库执行器、游戏引擎和网络服务都常使用这种思路。

冷热分离也有代价：代码复杂度增加，对象关系不如原来直观。是否值得做，要看热路径字段访问比例和缓存压力。设计数据结构时，先写清哪个路径是热路径，哪个字段在热路径必读或必写。

6.4 内存分配和对象布局

小对象频繁分配会带来分配器开销、碎片、TLB 压力和缓存局部性问题。高性能系统常用 arena、对象池、批量分配或结构化生命周期来减少热路径分配。并发分配器还要避免全局锁和跨线程释放导致的远端缓存流量。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清晰、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

对象池和 arena 的核心收益不是“分配函数少调用几次”这么简单。它们把生命周期相近的对象放在相近内存中，减少元数据开销和碎片，提高遍历局部性，也让释放可以批量发生。并发环境下，还可以按线程或 NUMA 节点维护本地池，减少全局分配器锁和远端释放。

代价同样要写清楚。Arena 适合批量释放，不适合单个对象精确析构；对象池可能保留大量空闲内存；跨线程归还对象可能重新引入同步；自定义分配器如果没有测试，可能比通用分配器更难维护。高质量教材不能只说“用内存池更快”，必须说明适用的生命周期形状。

6.5 布局与并发

多线程程序的数据布局还要考虑写共享。线程本地数据应尽量分开，跨线程只读数据可以共享，跨线程写数据要减少同一 cache line 上的冲突。并行归约、线程池队列和日志缓冲区都常常需要为每个线程准备独立槽位。

6.6 结构体大小和 ABI

结构体字段顺序影响 padding 和总大小。较大的结构体会降低缓存密度，也会增加拷贝成本。公共 ABI 中随意调整字段顺序可能破坏二进制兼容，因此高性能库常把内部热结构和外部稳定接口分开。内部结构按性能布局，外部接口保持稳定语义。

6.7 布局重构流程

数据布局优化应按流程进行。第一步，确认热路径和访问矩阵。第二步，估算每轮实际需要读写的字节数。第三步，检查对象是否连续、是否跨页、是否存在共享写。第四步，设计一个只改变布局、不改变算法语义的对照版本。第五步，用相同输入和相同绑定策略测量时间、cache、TLB 和扩展曲线。第六步，评估代码复杂度是否值得。

很多布局优化失败，是因为同时改了太多东西：换容器、改算法、改线程数、改内存分配、改输入规模。这样即使变快，也不知道原因。教材实验必须强调单变量对照，让读者学会建立因果证据。

实验：实现 AoS 和 SoA 两个版本的字段求和。比较顺序访问和随机访问。再构造两个线程分别写相邻字段和分离字段的场景，观察 false sharing。

第 7 章

SIMD、指令级并行与向量化

SIMD 让一条指令处理多个数据 lane，是现代 CPU 单核吞吐的重要来源。指令级并行则让多个互不依赖的操作同时在不同执行资源上推进。二者共同决定很多数值循环的上限。

7.1 从标量语义到向量语义

向量化不是把代码表面改成更复杂的形式，而是确认多个元素之间没有依赖，可以被同一条向量指令处理。数组逐元素加法容易向量化，因为每个元素独立；前缀和不容易直接向量化，因为后一个结果依赖前一个结果。

编译器自动向量化需要知道别名关系、对齐、循环边界和数据依赖。使用 `std::span` 表达连续区间有助于接口清晰，但编译器仍可能因为别名保守而放弃向量化。查看优化报告和汇编，比猜测更可靠。

判断循环能否向量化，要先写出每次迭代之间的关系。如果第 `i` 次迭代只读写第 `i` 个位置，通常容易向量化；如果第 `i` 次迭代读取第 `i-1` 次写出的结果，就存在循环携带依赖；如果每次迭代根据数据写入不同位置，就可能有冲突写；如果循环内部调用不可内联函数，编译器可能无法证明副作用边界。向量化首先是语义问题，其次才是指令问题。

7.2 别名和 restrict 思维

两个指针可能指向同一片内存时，编译器必须保守。例如 `a[i]=b[i]+c[i]` 中，如果 `a`、`b`、`c` 可能重叠，编译器要保证重叠情况下结果仍正确，这可能阻碍向量化。C++ 没有标准 `restrict` 关键字，但接口设计可以减少别名不确定性，例如使用不重叠容器、清晰文档和局部拷贝。

手写 SIMD 前，应该先让标量代码容易被编译器理解。清晰的循环边界、连续数据、少分支、少别名，是自动向量化的基本条件。

7.3 多累加器和延迟隐藏

即使不用 SIMD，多个独立累加器也能提高吞吐。单累加器形成长依赖链，每次加法依赖上一次结果；多个累加器把依赖链拆开，让乱序执行有更多可调度操作。SIMD 内核中也常见多个向量累加器，目的相同。

这就是并行归约在单线程内部也有优化空间的原因。先分成多个局部和，再合并，不仅适用于多线程，也适用于单核流水线。

Listing 7.1 多个累加器拆开依赖链

```
1 std::int64_t s0 = 0;
2 std::int64_t s1 = 0;
3 for (std::size_t i = 0; i + 1 < values.size(); i += 2) {
4     s0 += values[i];
5     s1 += values[i + 1];
6 }
7 const std::int64_t sum = s0 + s1;
```

这段代码的意义不是只用两个累加器，而是展示依赖链拆分。真实内核会根据执行端口、寄存器数量和向量宽度选择更多累加器。累加器太少隐藏不了延迟，太多会增加寄存器压力。

7.4 尾部处理和对齐

向量宽度通常不能整除数组长度，因此要处理尾部元素。常见方法是标量尾循环、掩码加载和补齐填充。对齐可以减少某些加载成本，但现代 CPU 对非对齐连续访问已经比较友好；真正要避免的是跨 cache line、跨页和随机访问。

尾部处理是性能代码里常见的正确性漏洞。主循环只处理完整向量宽度，剩余元素必须保证不丢、不重复、不越界。手写 intrinsics 时，尾部可以用标量循环、mask load/store 或提前 padding。选择哪种方式要看 ISA 支持、数据类型、越界读是否合法、输出是否允许覆盖 padding。教材示例宁愿保守清晰，也不能为了展示技巧隐藏边界条件。

常见误区

不要为了让 SIMD 主循环漂亮而忽略尾部。真实库里的很多错误，不发生在主循环，而发生在长度为 0、1、向量宽度减 1、刚好跨页或输入输出重叠的边界。

7.5 SIMD 与线程的关系

SIMD 和多线程不是替代关系。SIMD 提高单核吞吐，多线程使用多个核心。高性能程序通常先让单线程内核足够高效，再扩展到多线程。否则多线程只会复制低效内核，并更快撞上内存带宽。

7.6 数值语义

向量化可能改变浮点运算顺序，从而改变舍入误差。整数加法在不溢出的数学模型下更容易重排，浮点加法不严格满足结合律。编译器的 fast-math 选项会放宽数值语义，可能提高性能，也可能改变边界行为。工程中必须明确是否允许这种变化。

7.7 自动向量化到手写 SIMD

优化顺序通常是先写清晰标量代码，再查看编译器是否自动向量化，再决定是否手写 intrinsics。自动向量化失败时，先看原因：别名不明确、循环边界复杂、函数调用副作用、数据依赖、类型转换、未对齐访问还是成本模型认为不值得。只有当标量语义已经清楚、数据布局稳定、热点足够重要、自动向量化不足时，手写 SIMD 才值得。

手写 SIMD 的维护成本很高。它引入 ISA 分发、不同宽度实现、尾部处理、测试矩阵和可移植性问题。生产库通常会把通用标量 reference、自动向量化版本和手写 ISA 版本分开，并用同一组 correctness tests 验证。第二册讲 SIMD 的目标不是让读者到处写 intrinsics，而是让读者知道什么时候应该追求向量吞吐，什么时候瓶颈根本不在这里。

7.8 向量化和内存带宽

SIMD 提高的是每条指令处理的数据量，但它不能凭空增加内存带宽。对于流式数组求和，如果单线程已经被内存带宽限制，更宽的 SIMD 可能只减少指令数，不显著提高总时间。对于算术强度高、数据能复用的内核，SIMD 才可能接近计算峰值。判断前必须回到 Roofline：这个内核到底缺算力，还是缺数据。

实验：用编译器优化报告观察一个数组加法循环是否向量化。再写单累加器和四累加器求和，比较性能。记录编译选项和 CPU 指令集。

第 8 章

典型计算内核模式

高性能计算里反复出现一些内核模式：map、reduce、scan、stencil、矩阵块计算、直方图、排序、图遍历和队列驱动任务。掌握这些模式，比记住某个库函数更重要。

8.1 Map 与 Reduce

Map 对每个元素独立变换，通常容易并行和向量化。Reduce 把多个元素合成一个结果，存在合并顺序问题。整数求和通常结合律明确，浮点求和则会因为舍入误差导致不同合并顺序产生不同结果。

并行 reduce 的基本形态是每个线程处理一段输入，写入线程本地 partial，然后合并。这个结构减少共享写，也让每个线程顺序扫描自己的数据块。Compute Systems Labs 的 `parallel_sum` 就是这个最小模型。

Reduce 的关键不是“把加法拆给多个线程”这么简单，而是确定合并操作是否满足结合律、是否允许重排、partial 放在哪里、合并树长什么样。整数求和如果可能溢出，重排也可能改变 C++ 语义；浮点求和即使不溢出，也会改变舍入；字符串拼接满足语义结合时，内存分配可能成为主要成本。因此每个 reduce 都要同时写正确性模型和成本模型。

8.2 Map 的融合

多个 map 连在一起时，如果每一步都写出中间数组，会增加内存流量。融合把多个逐元素变换合在一次循环里，减少中间读写。融合不是总是好事：融合后循环体变大，寄存器压力可能增加，分支也可能更复杂。是否融合，要看减少的内存流量是否超过新增的执行压力。

8.3 Stencil 与邻域计算

Stencil 计算每个输出元素时读取邻域元素，例如图像卷积、有限差分 and 网格模拟。它的关键是复用邻域数据，避免重复从内存加载。分块、滑动窗口和边界处理是主要问题。

Stencil 的边界处理会影响代码形状。可以把内部区域和边界区域分开，让内部热循环没有复杂分支；也可以用 padding 或 ghost cell 让边界访问统一。前者代码路径多但热循环干净，后者内存多一点但逻辑规律。并行 stencil 还要处理块之间的 halo 数据，单机上表现为相邻块共享边界，分布式上表现为节点之间交换 halo 消息。

8.4 矩阵和块算法

矩阵计算展示了数据复用的重要性。朴素矩阵乘的数学公式很简单，但访问顺序会决定是否复用缓存。分块算法把矩阵切成能放入缓存的小块，让一个块内的数据被多次使用。这个思想不仅用于矩阵，也用于排序、join、图处理和文件扫描。

块大小不是越大越好。太小会增加循环和调度开销，太大又放不进缓存。实际工程中常用 benchmark 或自动调优选择块大小，并针对不同 CPU 做参数化。

8.5 直方图和冲突更新

直方图看似简单：读一个元素，更新一个桶。但多个线程可能更新同一个桶，导致锁竞争或原子争用。常见优化是线程本地直方图，最后合并。这个模式体现了一个原则：把高频共享写变成低频合并。

直方图还会遇到数据倾斜。如果 key 分布均匀，线程本地桶很好；如果大量输入集中到少数桶，全局原子会退化，分片后合并也可能在少数桶上集中。工程实现常按线程、NUMA 节点或缓存块建立局部桶，并在合并阶段使用连续扫描。若桶数量很大，本地直方图会占用大量内存，这时要在内存占用和冲突减少之间折中。

8.6 图遍历

图遍历通常局部性差、分支多、负载不均。压缩邻接表、按层处理、前沿队列和方向优化，都是为了改善访问模式和任务分配。图算法提醒我们，不是所有问题都能靠 SIMD 和连续数组轻松解决。

图遍历的性能问题往往同时包含三件事：邻接表访问随机，顶点度数不均，前沿大小变化剧烈。BFS 在某些层只有少量顶点，线程不够忙；在某些层前沿巨大，访问又非常分散。方向优化 BFS 会在 top-down 和 bottom-up 之间切换，目的就是根据前沿大小改变访问模式。这个例子说明，高性能算法不是固定套一个并行模板，而是随数据形状选择执行策略。

8.7 排序和分区

排序是数据系统的核心内核。快速排序、归并排序、基数排序和样本排序在不同场景下表现不同。并行排序的难点不只是比较次数，还包括分区均衡、临时空间、cache 行为和跨线程合并。分布式排序还会引入 shuffle 和热点分区。

8.8 内核选择清单

遇到一个新计算内核，可以按清单分析。第一，输入输出是否连续，是否能按块切分。第二，每个元素工作量是否接近，是否存在数据倾斜。第三，是否有跨元素依赖，依赖能不能分层处理。第四，写入位置是否唯一，是否会冲突。第五，是否能把全局共享写改成本地写加合并。第六，算术强度大概是多少，瓶颈更像计算还是内存。第七，单机算法扩展到分布式时，哪一步会变成 shuffle 或远程通信。

这份清单会在后面的并行算法和分布式计算里反复出现。它让读者从“我知道几个算法名字”转向“我能根据数据和依赖形状选择执行方式”。

核心思想

分析计算内核时先问四个问题：每个元素读写多少字节，是否存在跨元素依赖，写入是否冲突，工作量是否均衡。

实验：在 Compute Systems Labs 中扩展一个线程本地直方图。先写全局 mutex 版本，再写线程本地版本，比较扩展性，并解释差异来自锁竞争还是 cache line 迁移。

第零部分

多线程、同步与内存模型

第 9 章

线程、调度与亲和性

线程是操作系统调度的执行实体。多线程程序能利用多核心，但也引入调度、同步、内存可见性和资源竞争。理解线程，必须同时看用户态代码和内核调度。

9.1 线程不是核心

创建八个线程不等于拥有八个物理核心。硬件可能有 SMT，一个物理核心暴露两个逻辑 CPU；系统里还运行着其他进程；线程可能被迁移到不同核心。线程迁移会破坏缓存热度，过多线程会增加上下文切换。

CPU-bound 程序通常不应远多于核心数，I/O-bound 程序可以有更多并发等待。判断依据不是经验数字，而是运行时是否在计算、等待锁、等待 I/O 或被调度器抢占。

9.2 用户线程和内核线程

C++ 的 `std::thread` 通常映射到操作系统线程。操作系统调度的是内核可见的执行实体，它拥有寄存器上下文、栈、调度状态和优先级。用户态协程则不同，协程切换通常不进入内核，它把等待点显式暴露给运行时。协程适合组织大量 I/O 等待，不会自动让 CPU 密集代码并行。

这一区分很重要。把 CPU 密集任务放进单线程协程事件循环，不会使用多个核心；把大量 I/O 等待都变成内核线程，可能造成过多栈内存、调度成本和上下文切换。运行时设计要把计算并行和等待并发分开。

9.3 调度和上下文切换

上下文切换需要保存和恢复执行状态，可能破坏 cache 和 TLB 局部性。频繁阻塞、线程过多、锁竞争和系统调用都可能增加切换。性能计数器里的 context-switches 和 migrations 可以帮助定位这类问题。

Linux CFS 调度器大体追求公平，它会根据虚拟运行时间选择任务。实时调度策略和普通调度策略不同，容器和 cgroup 也会改变 CPU 配额。benchmark 时若机器上有其他负载，或者进程受 cgroup 限制，结果会明显波动。严谨实验应记录负载、CPU governor、容器限制和是否独占机器。

调度器不是只在“线程很多”时才重要。即使线程数等于核心数，线程也可能因为缺页、锁等待、I/O、信号、中断、抢占和 cgroup 配额离开 CPU。对于低延迟系统，一次迁核可能让热缓存失效；对于吞吐系统，频繁上下文切换会减少有效计算时间。性能报告如果只写线程数，不写线程是否真正持续运行在 CPU 上，就缺少关键证据。

Linux 调度源码阅读不建议一开始追完整策略。更好的入口是从一个用户态现象出发：为什么 `perfstat` 里 context-switches 增加，为什么线程会 migration，为什么绑定 CPU 后波动变小，为什么 cgroup 限额下吞吐呈周期性抖动。围绕这些问题，再去看调度实体、runqueue、唤醒路径和上下文切换路径，读出的内容才有边界。

9.4 阻塞、睡眠和自旋

线程等待锁时，可以自旋，也可以睡眠。自旋避免进入内核，适合等待时间极短的场景；睡眠释放 CPU，适合等待时间较长的场景。很多锁实现会先短暂自旋，再进入 `futex` 等内核等待路径。这样低竞争时快，高竞争时不至于烧满 CPU。

选择自旋还是阻塞，本质是在 CPU 时间和唤醒延迟之间取舍。如果临界区很短且核心充足，自旋可能好；如果锁持有者被抢占或等待 I/O，自旋会浪费大量周期。不要在没有测量的情况下把自旋当成高性能。

`futex` 是理解 Linux 用户态锁的重要入口。常见 `mutex` 在无竞争时尽量只走用户态原子操作；竞争严重时，线程通过 `futex` 进入内核睡眠，锁释放时再唤醒等待者。这样做把低竞争 fast path 和

高竞争 blocking path 分开。读锁性能时，应区分用户态自旋成本、futex 睡眠成本、唤醒成本和被唤醒后重新竞争锁的成本。

9.5 线程亲和性

线程亲和性让线程倾向或固定在某些 CPU 上运行。它可以改善缓存局部性和 NUMA 放置，也可能降低调度器自由度。使用亲和性前要理解拓扑：物理核心、SMT 线程、NUMA 节点和共享缓存层级。

亲和性实验要有对照。只绑定线程不绑定内存，可能仍然访问远端页面；只绑定内存不绑定线程，线程迁移后也可能变成远端访问。合理实验至少比较未绑定、只绑线程、只绑内存、线程和内存都按节点绑定几种情况。若平台没有 NUMA，也应记录物理核心和 SMT 拓扑，避免把两个重负载线程绑到同一个物理核心的两个逻辑 CPU 上后误判算法慢。

9.6 线程生命周期

频繁创建销毁线程成本高，也让任务调度不可控。线程池把线程生命周期和任务生命周期分离，适合大量短任务。但线程池不是万能的，如果任务太小，调度成本会超过计算成本；如果任务阻塞，线程池可能被耗尽。

9.7 线程局部存储

线程局部存储让每个线程拥有独立对象，适合缓存临时缓冲、统计计数和避免共享写。它可以减少锁竞争，但也可能增加内存占用，并让跨线程汇总更复杂。线程池里使用线程局部对象时，还要注意任务迁移：同一个逻辑请求的不同阶段可能被不同 worker 执行，不能把请求状态误放到线程局部。

9.8 线程池里的调度语义

线程池把固定 worker 暴露成任务执行资源，但它不会自动解决调度语义。CPU 密集任务应该限制并发度，避免超过核心数太多；阻塞 I/O 任务如果占住 worker，可能让计算任务饥饿；任务内部再提交子任务并等待，可能造成线程池死锁。设计线程池时，要明确任务是否允许阻塞、是否允许嵌套提交、是否支持 work helping、是否需要独立 I/O 池。

线程池还会改变线程局部数据的含义。线程局部缓冲属于 worker，不属于请求。请求如果跨多个任务执行，就不能假设后续阶段还能访问同一个线程局部对象。运行时应把“worker 本地缓存”和“请求上下文”分成两个概念。

核心思想

线程设计的核心不是“开几个线程”，而是决定哪些状态线程本地，哪些状态共享，哪些共享状态需要同步，哪些等待应该阻塞或异步化。

实验：把并行归约的 worker 数从 1 增加到逻辑 CPU 数的两倍。记录时间、context-switches 和 migrations。解释从哪个点开始扩展性变差。

第 10 章

锁、条件变量与信号量

锁的作用是保护共享状态，让临界区在同一时间只被一个线程执行。锁不是性能敌人，也不是正确性的全部。正确使用锁，需要明确保护对象、锁顺序、临界区长度和等待条件。

10.1 互斥锁保护什么

每把锁都应该有清晰的保护范围。一个队列锁保护 head、tail、size 和 buffer 状态；一个计数器锁保护 value；一个对象锁保护对象不变量。锁如果没有对应的不变量，就容易出现“看起来加锁了但仍然错”的情况。

临界区应尽量短，但不能为了短而破坏不变量。把状态检查放在锁外，可能引入竞态；把耗时 I/O 放在锁内，会扩大阻塞范围。工程上要在正确性和粒度之间找到边界。

Listing 10.1 锁保护队列不变量

```
1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->values_.push_back(value);
6     }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };
```

这段代码的关键不是 `push_back` 这一行，而是锁和不变量的关系：`values_` 的内部结构不能被多个线程同时修改。如果未来再加入 `size_`、`head_` 或统计字段，它们也必须在同一把锁保护下保持一致。

写锁代码时，应先写“锁保护的不变量”，再写代码。比如有界队列的不变量可以是：`0<=size<=capacity`，`head` 和 `tail` 总是落在环形数组范围内，`size==0` 表示不能 pop，`size==capacity` 表示不能 push。互斥锁保护的是这些关系，不只是保护某一个变量。若 `size` 在锁内改，`head` 在锁外改，这个不变量就被拆碎了。

10.2 锁粒度

粗粒度锁容易证明正确，但并发度低。细粒度锁提高并发度，却增加锁顺序、死锁和状态拆分复杂度。分片锁是一种常见折中：按 key、队列分段或线程组拆成多把锁，让无关操作并行，热点操作仍受保护。

锁粒度应从访问模式出发。如果所有请求都访问同一个热点 key，分片没有帮助；如果请求天然分散，分片锁可以显著减少竞争。锁优化前要先测量等待时间和竞争比例。

10.3 条件变量

条件变量用于等待某个状态变真。它必须和互斥锁配合，并在循环里检查谓词。原因是线程可能被虚假唤醒，也可能在被唤醒后发现条件已经被其他线程消费。正确模式是“持锁检查条件，不满足则等待，醒来继续检查”。

Listing 10.2 条件变量等待谓词


```

1 std::unique_lock<std::mutex> lock(this->mutex_);
2 this->not_empty_.wait(lock, [this]() {
3     return !this->values_.empty();
4 });
5 const std::int32_t value = this->values_.front();

```

这里谓词属于受锁保护的状态。通知本身不携带数据，数据在共享状态里。若不用循环检查谓词，就可能在虚假唤醒或竞争消费后读取空队列。

条件变量最重要的句子是：等待的是谓词，不是通知。通知可能在等待前发生，也可能唤醒多个线程，也可能出现虚假唤醒。只要谓词在锁保护下检查，程序就正确；只依赖“我收到过一次 notify”，程序就脆弱。生产者消费者队列里，`not_empty` 的谓词是队列非空，`not_full` 的谓词是队列未滿。两个谓词对应同一组受锁保护状态。

10.4 信号量和资源计数

信号量表示可用资源数量。它适合限制并发度、实现 bounded buffer 或控制连接池。信号量和互斥锁不同：互斥锁保护状态的一致性，信号量表达资源额度。混用时要小心锁顺序，避免死锁。

10.5 死锁和锁顺序

死锁通常需要互斥、持有并等待、不可抢占和循环等待四个条件。破坏循环等待的常见方法是规定全局锁顺序。复杂系统里，锁顺序文档和锁级别检查很重要，因为死锁往往在低概率路径中出现。

10.6 优先级反转和 convoy

优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，导致高优先级线程无法前进。锁 convoy 则是多个线程排队等待同一锁，锁释放后唤醒和抢占成本让系统吞吐下降。这些问题说明锁不仅是用户态对象，也和调度器行为相互作用。

工程系统要避免在锁内做不可控工作，例如磁盘 I/O、网络调用、复杂回调和用户插件。锁内执行时间越不可预测，越容易出现尾延迟问题。

10.7 锁的性能路径

mutex 通常有三条路径。第一条是无竞争 fast path，用户态原子操作成功，成本较低。第二条是短暂竞争路径，线程可能自旋几轮，等待锁持有者很快释放。第三条是阻塞路径，线程进入内核等待，涉及 futex、调度和唤醒。性能分析要判断当前主要走哪条路径。一个低竞争 mutex 没必要改成复杂无锁结构；一个长期进入阻塞路径的热点锁，则需要拆分状态、缩短临界区或改变算法。

临界区长度不只由代码行数决定。锁内分配内存、写日志、调用用户回调、访问远端 NUMA 内存、发生 page fault、触发异常路径，都会让持锁时间变长。锁优化报告应列出锁内可能的慢操作，而不只是说“这个锁很粗”。

10.8 读写锁和 RCU 直觉

读多写少场景中，读写锁允许多个读者并发，但写者需要独占。它适合读临界区不太长、写入不太频繁的结构。若读者很多且读路径必须极轻，RCU 的思想更进一步：读者在不阻塞写者的情况下读取旧版本，写者发布新版本后，等所有旧读者离开再回收旧对象。

RCU 的难点不在“读很快”这个口号，而在对象生命周期。旧对象什么时候没人再读，什么时候可以释放，如何处理写者之间的同步，都是必须证明的问题。后面讲无锁内存回收时，会看到相同主题。

常见误区

不要把锁竞争问题简单理解成“换成无锁”。如果临界区保护的是复杂不变量，无锁实现可能更难证明正确，也可能在高竞争下 CAS 失败更多。

实验：实现一个 bounded queue。先用 mutex 和 condition variable 写阻塞 push/pop，再记录高生产者数量下的等待时间。解释锁保护的状态和等待谓词。

第 11 章

原子操作与内存序

原子操作提供不可分割的读写或读改写，并参与线程间同步。内存序定义操作之间的可见性和重排约束。理解原子操作，必须区分原子性、顺序性和生命周期。

11.1 原子不是普通变量

`std::atomic` 的 `load`、`store`、`fetch_add`、`compare_exchange` 等操作会生成特殊指令或特殊约束。它们可以避免数据竞争，但不自动保证算法正确。一个原子计数器容易写，一个无锁队列需要处理节点发布、竞争失败、ABA 和回收。

低竞争下，原子操作可能比 `mutex` 轻；高竞争下，多个线程反复修改同一 `cache line`，原子也会扩展性很差。判断依据仍然是共享写频率和一致性流量。

11.2 常见内存序

`memory_order_relaxed` 只保证原子性，不建立跨变量同步。计数器统计常用 `relaxed`，因为只关心最终数值。`memory_order_release` 发布写入，`memory_order_acquire` 获取发布的结果，二者配合可建立 `happens-before`。`memory_order_seq_cst` 提供更强的全局顺序，但成本和约束也更强。

选择内存序的原则是从语义出发，而不是从性能愿望出发。若一个线程写好对象后通过原子标志发布，写标志应 `release`，读标志应 `acquire`。若只是统计事件数量，`relaxed` 足够。

Listing 11.1 `release/acquire` 发布数据

```
1 struct Shared {
2     std::int32_t data = 0;
3     std::atomic<bool> ready = false;
4 };
5
6 void producer(Shared& shared) {
7     shared.data = 42;
8     shared.ready.store(true, std::memory_order_release);
9 }
10
11 std::int32_t consumer(Shared& shared) {
12     while (!shared.ready.load(std::memory_order_acquire)) {
13     }
14     return shared.data;
15 }
```

`release store` 之前的普通写，在 `acquire load` 看到该 `store` 后，对 `consumer` 可见。这里不能把 `ready` 简单改成普通 `bool`，也不能把 `acquire/release` 无脑改成 `relaxed`；否则程序缺少跨线程发布关系。

理解 `acquire/release` 时，不要把它想成“刷新所有缓存”这样粗糙的动作。更准确的教材模型是：`release` 把之前的写放在发布点之前，`acquire` 在看到这个发布后，让之后的读写不能越过获取点，并建立 `happens-before`。具体硬件怎样实现，取决于架构强弱和编译器约束。C++ 程序只应依赖语言层面的同步关系。

11.3 Relaxed 的正确用途

Relaxed 常用于统计计数、采样标志、引用计数的某些阶段和不参与对象发布的指标。它保证原子变量自身不会数据竞争，但不保证其他变量的可见性。用 relaxed 写高性能代码时，必须能说清“我只需要这个原子变量自己的数值，不需要借它同步别的数据”。

例如请求计数器可以用 relaxed，因为最终统计只关心总次数。对象指针发布不能只用 relaxed，因为读到指针的线程还需要看到对象内部字段已经初始化。

11.4 CAS 循环

Compare-and-swap 读取旧值，若仍等于期望值就写入新值。无锁算法常用 CAS 循环处理竞争。CAS 失败不表示错误，而是说明其他线程已经修改状态，需要重新读取并尝试。

CAS 循环必须考虑进度。低竞争时很快，高竞争时可能大量失败。复杂结构还要考虑 ABA：一个位置从 A 变 B 又变回 A，CAS 看到 A 可能误以为没有变化。解决方法包括版本号、hazard pointer、epoch 回收等。

CAS 循环还必须考虑循环体里是否做了不可重复副作用。失败重试意味着循环体可能执行多次，如果每次尝试都分配内存、写日志、修改外部状态或增加不可回滚计数，就会产生额外语义问题。常见写法是先准备候选新状态，在 CAS 成功后才执行对外可见副作用；失败时只更新局部期望值并重试。

11.5 CAS 的 weak 和 strong 版本

`compare_exchange_weak` 允许伪失败，即使当前值等于期望值也可能失败，因此通常放在循环里。`compare_exchange_strong` 不允许伪失败，更适合不在循环里的单次尝试。很多平台上二者生成代码可能接近，但语义差异必须理解。

CAS 还有一个容易忽视的行为：失败时会把当前实际值写回 `expected` 参数。这让循环可以基于新值继续尝试，但也容易让初学者误用 `expected`。写无锁代码时，要把成功内存序和失败内存序都写清楚。

11.6 内存屏障和硬件

不同架构的内存模型不同。x86-64 相对强，ARM 和 POWER 更弱。C++ 内存序提供跨平台抽象，编译器和硬件会根据目标架构生成必要约束。不要因为某段代码在 x86 上“看起来没问题”，就认为它是正确的并发程序。

11.7 原子与缓存一致性

原子读改写通常会让目标 cache line 进入独占修改路径。多个核心同时对一个原子变量做 `fetch_add`，实际上是在争夺同一条 cache line 的所有权。内存序越强，编译器和硬件重排空间越小；竞争越强，一致性流量越大。

这就是为什么高性能计数常用分片计数器。每个线程或每个核心写自己的本地计数，读取总数时再合并。它牺牲实时精确读取成本，换取写入热路径扩展性。

11.8 内存序选择流程

选择内存序可以按问题反推。第一，是否只需要原子变量自身不撕裂、不数据竞争。如果是统计计数，通常 relaxed 足够。第二，是否通过某个原子变量发布了其他普通数据。如果是，需要 `release/acquire` 或更强同步。第三，是否需要多个线程观察到一个全局一致顺序。如果确实需要，才考虑 `memory_order_seq_cst`。第四，是否可以用 mutex 表达更清晰的不变量。如果共享状态复杂，mutex 往往比一组难懂的原子更可靠。

内存序不是性能调参旋钮。把 `memory_order_seq_cst` 改成 relaxed 之前，必须写出被保留的同步关系和被放弃的顺序关系。若写不出来，说明程序设计还没清楚到可以安全降级。

11.9 x86 和 ARM 的差异

x86-64 的内存模型相对强，很多 acquire/release 在硬件指令上看起来很轻；ARM 等架构更弱，可能需要更显式的屏障。可移植 C++ 代码不能因为在 x86 上测试通过，就省略必要内存序。教材示例必须按 C++ 语义写正确，而不是按某台机器的偶然行为写正确。

实验：比较 mutex counter 和 atomic counter。在低线程数和高线程数下记录耗时。解释 relaxed 为什么适合这个计数器，也说明它为什么不能替代对象发布同步。

第 12 章

无锁数据结构

无锁数据结构的目标不是“没有同步”，而是在不使用互斥锁阻塞线程的情况下维护共享状态。它通常依赖原子操作、CAS、内存序和严格的对象生命周期管理。学习无锁结构，要先学习进度保证和失败模式。

12.1 进度保证

Blocking 算法可能因为一个线程持锁停顿而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常更难实现。Obstruction-free 只在无竞争时保证进展。

这些术语不是装饰。一个队列在压力下 CAS 反复失败，可能仍然 lock-free，但尾延迟很差。工程上需要结合竞争程度、临界区复杂度和可维护性选择方案。

进度保证要和系统目标匹配。低延迟交易路径可能愿意为 wait-free 付出巨大复杂度；普通后台队列通常 mutex 就足够；高吞吐日志管线可能只需要 SPSC ring buffer；线程池全局队列可能更适合分片队列加工作窃取。不要把 lock-free 当成荣誉标签，它只是进度性质的一种选择。

12.2 无锁栈和 ABA

Treiber stack 是经典无锁栈。push 读取 head，把新节点 next 指向旧 head，再 CAS head。pop 读取 head 和 next，再 CAS head 到 next。问题在于 ABA：head 可能从 A 变成 B 又变回 A，CAS 看见 A 成功，但 A 的生命周期可能已经变化。

解决 ABA 需要额外机制。版本标签把指针和计数器一起比较；hazard pointer 让线程声明自己正在访问的节点；epoch reclamation 延迟释放节点直到所有线程离开旧 epoch。无锁算法难点往往不在 CAS 本身，而在内存回收。

```
Treiber push shape:
load old head
new node next = old head
CAS head from old head to new node

Treiber pop shape:
load old head
load next from old head
CAS head from old head to next
```

这张形态图比直接贴完整代码更重要。读者必须先看出线性化点在哪里：push 和 pop 都在 CAS 成功那一刻生效。CAS 之前读到的状态只是一次尝试，失败后不能假装操作已经发生。

12.3 内存回收为什么困难

带锁结构中，线程持有锁时可以知道没有其他线程正在修改结构。无锁结构没有这个简单边界。一个线程从 head 读到节点指针后，可能还没来得及读取 next，另一个线程已经 pop 并释放了该节点。第一个线程随后访问这个指针，就会出现 use-after-free。

垃圾回收语言可以把节点释放推迟到不可达之后，C++ 程序必须自己设计回收协议。Hazard pointer 的思路是线程公开声明“我可能访问这个节点”，释放者扫描这些声明，确认没有线程持有后再释放。Epoch 的思路是把释放延迟到所有线程都离开旧时代之后。二者都有开销，也都有使用约束。

Hazard pointer 更精确，但每个线程需要发布自己正在保护的指针，释放者需要扫描 hazard 集

合。Epoch 回收批量化更强，读路径常常更轻，但如果某个线程长时间停留在旧 epoch，回收会延迟，内存占用可能增长。RCU 也是类似思想：读者进入读侧临界区，写者等待 grace period 后回收旧版本。不同机制的共同点是：释放不能只看“节点已经从结构中摘掉”，还要看“是否还有线程可能持有旧指针”。

12.4 ABA 的具体例子

假设栈顶是 A，线程 T1 读取 head=A，准备把 head 改成 A->next。此时 T2 pop A，又 pop B，然后把 A push 回栈顶。T1 再 CAS head，从 A 改成旧的 A->next。CAS 看到 head 仍然是 A，于是成功，但这个 A 已经不是 T1 当初理解的结构关系。版本标签通过让 head 变成“指针加版本号”，使 A 回来时版本不同，从而让 CAS 失败。

12.5 环形队列

有界环形队列适合固定容量、低分配、缓存友好的场景。单生产者单消费者队列可以用较简单的 head/tail 原子实现；多生产者多消费者队列需要为槽位维护序号或状态，避免多个线程同时写同一槽位。

环形队列的性能来自固定内存、连续布局 and 减少分配。但如果所有生产者争用同一个 tail，仍然会有原子热点。分片队列、每线程队列和工作窃取可以缓解热点。

12.6 SPSC、MPSC 和 MPMC

队列难度取决于生产者和消费者数量。SPSC 只有一个生产者和一个消费者，head 只被消费者写，tail 只被生产者写，可以用较轻的同步。MPSC 有多个生产者争用 tail，消费者独占 head。MPMC 两端都有竞争，通常需要更复杂的槽位序号或链表算法。

设计队列时先问清使用场景，不要直接上最复杂的 MPMC。日志线程、音频缓冲和管线阶段之间常常只需要 SPSC；线程池全局队列可能需要 MPMC；工作窃取运行时则常用每 worker deque 加窃取操作。

SPSC ring buffer 是学习无锁队列的最佳起点。生产者独占 tail，消费者独占 head，因此每个索引只有一个写者；双方通过 acquire/release 观察对方进度。MPSC 和 MPMC 复杂得多，因为多个线程会争用同一个端点，需要 CAS、槽位状态或序号。教材顺序应从 SPSC 到 MPSC 再到 MPMC，而不是一开始贴一个难以证明的万能队列。

12.7 线性化点

并发数据结构的线性化点，是操作在逻辑上生效的那一瞬间。带锁结构的线性化点通常在锁内某次状态修改；无锁结构的线性化点通常是成功 CAS。若无法指出线性化点，就很难证明结构正确。

例如无锁栈 push 的线性化点是 CAS head 成功。CAS 失败的尝试没有生效，必须重试。测试无锁结构时，不能只看最终数量，还要考虑每个操作是否能排列成某个合法串行顺序。

线性化点还帮助写测试。对于栈，可以记录每个 push 的值和每个 pop 的返回，检查是否存在某个合法串行顺序；对于队列，要检查 FIFO 语义是否在并发交错下仍成立；对于计数器，要检查每次成功自增是否唯一。压力测试只能增加信心，不能替代线性化论证。无锁结构没有明确线性化点，通常就不该进入生产代码。

12.8 何时不用无锁

如果共享状态复杂、竞争不高、临界区很短，mutex 可能更清晰也足够快。如果需要条件等待，锁加条件变量往往比忙等 CAS 更节能。无锁结构适合明确的高频热路径，且必须有严格测试、压力测试和代码审查。

12.9 工程验收标准

无锁结构的验收标准应比普通容器更高。第一，要有清晰的不变量、线性化点和进度保证说明。第二，要说明内存回收策略，不能留下 use-after-free 风险。第三，要有单线程 reference 行为测试、并发压力测试、边界容量测试和关闭语义测试。第四，要有性能对照，证明它在目标竞争模式下确

实优于 mutex 或分片锁。第五，要说明不适用场景，例如高竞争 CAS 风暴、需要阻塞等待或对象生命周期复杂。

如果这些条件做不到，就应该优先使用更简单的同步结构。复杂无锁代码的维护成本非常高，错误通常低概率且难复现。严肃工程里，简单正确经常比炫技更有价值。

实验：阅读 Compute Systems Labs 的 bounded queue，写出 mutex 版本保护的不变量。再设计一个 SPSC ring buffer 的接口，说明哪些状态可以用 relaxed，哪些位置需要 acquire/release。

第零部分

并行算法与运行时

第 13 章

归约、扫描与分区

并行算法的核心是处理依赖。归约、扫描和分区是许多系统的基础操作：数据库聚合、日志统计、图计算、排序、渲染和科学计算都会用到它们。

13.1 归约

归约把多个输入合并成一个输出。并行归约的基本策略是局部归约加全局合并。它的性能取决于输入划分、局部性、合并成本和结果类型。整数加法简单，浮点加法要考虑精度和顺序，字符串拼接则可能有大量分配。

归约优化通常先避免共享写。每个线程写自己的 `partial`，最后串行或树形合并。树形合并可以减少合并深度，也更适合分布式系统。

归约的第一步是写出代数性质。操作是否有单位元，是否满足结合律，是否满足交换律，是否允许改变顺序。整数最小值、最大值、按位或通常容易并行；浮点求和需要说明误差模型；哈希合并需要说明冲突处理；对象聚合需要说明生命周期和分配。没有这些语义前提，谈并行归约是不完整的。

归约的第二步是写出内存布局。`partial` 结果应避免 `false sharing`，通常每个 worker 一个对齐槽位。若 `partial` 很大，可以按 NUMA 节点或缓存块组织，避免最后合并阶段变成单线程长尾。归约不是只有“线程本地加一下”这一种形式，它可以是线性合并、二叉树合并、分层合并或流水线合并。

13.2 扫描

扫描也叫 `prefix sum`，输出每个位置之前元素的累计结果。它比归约更难，因为每个输出都依赖前缀。并行扫描通常分两阶段：先对每个块做局部扫描，得到块总和；再扫描块总和；最后把块前缀加回块内结果。

扫描是很多算法的 `building block`，例如 `stream compaction`、并行分配输出位置、基数排序和图前沿构造。理解扫描，可以帮助读者把“有顺序依赖”的问题拆成局部和全局两层。

扫描比归约更能体现并行算法设计。表面上，每个输出都依赖前面所有输入，似乎完全串行；但把输入分块后，每个块内部可以先做局部扫描，块总和再做一次小规模扫描，最后把块前缀加回去。这个套路说明，很多“看起来串行”的依赖，可以拆成局部依赖和块间摘要。分布式系统里的很多计算也是这个形状：节点内先处理，节点间交换摘要，再回填结果。

13.3 分区

分区把元素按谓词或 `key` 分到不同区域。并行分区要先统计每个线程每个桶的数量，再计算全局偏移，最后写入输出。这与扫描密切相关。直接让多个线程 `push` 到共享 `vector`，会引入锁或原子热点。

分区的关键是先分配写入位置，再并行写入。以 `copy_if` 为例，朴素并行写法可能让所有线程对输出 `vector` 做 `push`，需要锁或原子。更好的写法是每个线程先统计自己保留多少元素，对这些计数做扫描得到每个线程的输出起点，然后每个线程写入自己的连续区间。这样写入没有冲突，输出也保持可解释顺序。

13.4 负载均衡

静态划分适合每个元素成本相近的场景。若元素成本差异大，静态划分会让某些线程早早结束，其他线程仍在工作。动态调度、任务队列和工作窃取可以改善负载均衡，但会增加调度开销。

负载均衡不是越动态越好。静态划分没有调度开销，局部性好，适合规则数组和均匀工作。动态队列能处理不均匀工作，但会增加同步和队列争用。工作窃取减少全局争用，但实现更复杂。设计时要估算任务成本方差：如果每个块成本接近，静态块划分更好；如果成本长尾明显，动态调度

才值得。

13.5 并行算法的正确性模板

本册要求每个并行算法都写出四个正确性问题。第一，串行语义是什么，reference 怎样实现。第二，并行切分后，每个 worker 负责哪些输入和输出。第三，局部结果怎样合并，合并顺序是否影响结果。第四，是否存在共享写、重复写、漏写或越界写。性能优化只能在这些问题回答之后进行。

这套模板能防止“并行版本跑得快但其实错”。很多并行 bug 不会在小输入暴露，因为线程交错少、边界简单、缓存状态稳定。测试必须覆盖空输入、极小输入、刚好一个块、多个块、不能整除块大小、重复 key、倾斜 key 和高线程数。

实验：在 Compute Systems Labs 中扩展 parallel `count_if`。先让每个线程本地计数，再合并。随后实现输出压缩：先统计、扫描偏移，再写输出数组。

第 14 章

任务运行时与工作窃取

线程是执行资源，任务是工作单位。把线程和任务分离，是构建可扩展运行时的关键。线程池、任务队列、工作窃取和任务图，是现代并行运行时的核心结构。

14.1 线程池

线程池创建固定数量或可控数量的工作线程，任务提交到队列，由工作线程取出执行。这样避免频繁创建线程，也便于控制并发度。线程池接口看似简单，难点在关闭、异常传播、任务等待、队列阻塞和避免死锁。

如果线程池里的任务再等待同一个线程池中的子任务，而工作线程数量不足，就可能发生饥饿。运行时设计必须考虑任务依赖，而不只是一个全局队列。

线程池的最小语义包括三件事：提交任务是否可能失败，关闭后队列中任务是否继续执行，调用 `wait` 或析构时是否等待所有任务完成。没有这些语义，线程池只是一个会运行函数的对象，不是可靠运行时。教材实现应先把这些边界写清，再谈性能。

14.2 Future 和等待依赖

Future 把“将来会完成的结果”变成对象。它适合表达异步任务，但如果 worker 线程在执行任务时同步等待另一个 future，就可能形成线程池内阻塞。更好的方式是依赖表达给运行时，让运行时在依赖满足后再调度后续任务，而不是占着 worker 等。

这也是任务图运行时的价值：等待不再消耗工作线程，依赖边决定任务何时 ready。构建系统和数据流引擎都大量依赖这种模型。

Future 的危险是把异步伪装成同步。用户看到 `future.get()` 很方便，但如果在 worker 线程里调用它，就可能把 worker 变成等待者。运行时可以通过 work helping 缓解：等待时主动执行其他 ready 任务；也可以要求任务不能阻塞等待，只能注册 continuation。两种设计都要明确写进接口语义。

14.3 任务粒度

任务太大，负载不均；任务太小，调度成本高。合理粒度通常需要根据输入规模、核心数、缓存块和调度开销选择。递归拆分算法常设 cutoff，小于某个规模就顺序执行。

任务粒度可以通过一个简单不等式判断：每个任务的计算时间应显著大于一次提交、入队、出队和唤醒的固定成本。若任务只做几十条指令，调度成本会吞掉收益；若任务运行几百毫秒，负载不均会造成长尾。实践中常把任务切到缓存友好的块大小，再用 benchmark 调整 cutoff。

14.4 工作窃取

工作窃取让每个线程拥有本地双端队列。线程优先从本地取任务，空闲时从其他线程窃取任务。这减少全局队列争用，并改善缓存局部性。常见策略是 owner 从底部 push/pop, stealer 从顶部 steal。

工作窃取的正确性和性能都依赖队列设计。Chase-Lev deque 是经典方案，但实现复杂，涉及原子、内存序和边界竞争。教学时先理解模型，再进入源码。

工作窃取还有一个重要性质：正常情况下 worker 访问自己的本地 deque，只有空闲时才访问别人的 deque。这把高频操作留在本地，低频窃取才跨线程同步。相比所有 worker 抢一个全局队列，它减少了共享热点。这个结构和 NUMA、分布式本地优先原则一致：先消费近处工作，再去远处拿工作。

14.5 本地性和窃取方向

工作窃取通常让 owner 按 LIFO 处理本地任务，让 stealer 从另一端 FIFO 窃取较老任务。LIFO 有利于缓存局部性，因为新产生的子任务往往访问相近数据；窃取较老任务则倾向拿到较大工作块，减少窃取频率。这个设计体现了运行时对局部性和负载均衡的折中。

14.6 任务图

很多计算不是简单队列，而是任务之间有依赖。任务图把节点表示任务，边表示依赖。运行时在依赖满足后调度任务。构建系统、数据处理引擎和图计算框架都大量使用任务图思想。

任务图需要维护 ready 计数。每个任务记录还有多少前驱未完成；当前驱完成时，后继计数减一；计数变为零，任务进入 ready 队列。这个模型避免 worker 阻塞等待依赖。它的难点在于任务失败、取消、异常传播和图生命周期：一个任务失败后，后继任务是跳过、取消还是继续运行，必须在运行时语义里说明。

14.7 取消和关闭

运行时必须定义关闭语义。提交中的任务是否接受，队列里的任务是否执行完，正在运行的任务是否可取消，异常如何传播，这些都要明确。没有关闭语义的线程池很容易在程序退出时丢任务、死锁或访问已销毁对象。

14.8 运行时观测指标

任务运行时不能只测总时间。还要观察队列长度、每个 worker 执行任务数、空闲时间、窃取次数、任务平均耗时、最长任务、等待依赖数量和关闭耗时。没有这些指标，负载不均和调度开销很难定位。一个任务图慢，可能是关键路径太长，不是 worker 不够；一个线程池慢，可能是任务太小，不是锁太慢。

这些指标也会指导分布式计算。单机里 worker 空闲，对应集群里 executor 空闲；单机里队列积压，对应集群里 stage backlog；单机里任务倾斜，对应分布式里的热点 key。运行时思维是从多线程走向分布式框架的桥。

实验：设计一个线程池接口，列出 submit、shutdown、wait 的语义。不要急着实现完整工作窃取，先实现固定线程数和全局队列，并写出关闭时不丢任务的测试。

第 15 章

I/O、异步与背压

计算系统不只做 CPU 运算。磁盘、网络、管道、日志和用户请求都会引入 I/O。I/O 的延迟和吞吐特征与 CPU 完全不同，因此运行时必须区分计算等待和 I/O 等待。

15.1 同步 I/O 和异步 I/O

同步 I/O 调用会让当前线程等待结果。少量线程处理少量 I/O 时简单可靠；大量并发连接或慢设备场景下，同步等待会浪费线程。异步 I/O 把提交和完成分离，让线程可以处理其他工作。

事件循环、回调、future、协程和 `io_uring` 都是组织异步 I/O 的方式。选择哪种方式，要看系统需要的并发数、延迟要求、代码复杂度和平台支持。

异步 I/O 的核心不是“换一种语法”，而是把等待从工作线程上移走。同步调用让当前线程停在系统调用或库调用里；异步模型把请求提交出去，完成时再处理结果。这样可以用较少线程管理大量等待。但异步也会引入状态机、取消、超时、错误传播和资源生命周期问题。若并发度很低，同步 I/O 可能更简单可靠。

15.2 背压

背压是系统告诉上游“我处理不过来”的机制。没有背压，队列会无限增长，内存耗尽，延迟失控。背压可以通过 bounded queue、令牌桶、限流、拒绝请求、降级和批处理实现。

并发系统里 bounded queue 的意义不只是节省内存，更是把过载变成可观察、可控制的状态。阻塞、返回错误或丢弃，都比默默无限排队更可管理。

背压必须一路传到真正的生产者。只在中间队列限长，但上游仍然无限接收请求，过载会换一个地方堆积。一个服务端如果 worker 队列满了，可以选择拒绝新请求、降低读取速度、返回重试、丢弃低优先级任务或触发降级。选择哪种方式取决于业务语义，但不能没有选择。

15.3 批处理

批处理用一次调度或一次 I/O 处理多个元素，摊薄固定成本。网络发送、日志写入、数据库提交和向量计算都常用批处理。批太小浪费固定成本，批太大增加延迟和缓存压力。

批处理要控制两个阈值：数量阈值和时间阈值。只按数量凑批，在低流量时可能让请求等太久；只按时间刷批，在高流量时可能批太小。很多系统使用“达到 N 条或等待 T 时间就发送”的策略。批处理还要考虑失败语义：一批里部分成功时如何重试，是否会重复执行，是否需要幂等 ID。

15.4 CPU 与 I/O 的边界

I/O 密集任务不应占满计算线程池。常见设计是分离 I/O 线程、计算线程和后台维护线程，或者使用异步运行时统一调度但明确资源限制。否则慢 I/O 会阻塞计算任务，计算任务也会延迟 I/O 完成处理。

15.5 队列是系统的压力表

队列长度、入队速率、出队速率和等待时间，是理解系统压力的基本指标。队列持续增长说明下游处理能力不足或上游没有背压；队列经常为空说明下游可能在等上游；队列长度锯齿状变化可能是批处理或周期性调度造成的。只看 CPU 使用率可能误判，因为系统可能 CPU 很空但卡在 I/O，也可能 CPU 很满但队列仍然增长。

有界队列实验要记录三种结果：生产者是否阻塞，消费者是否空闲，丢弃或拒绝是否发生。这样才能判断背压策略是否真的控制住过载。

实验：用 bounded queue 模拟生产者和消费者。让生产速度高于消费速度，观察队列大小和阻塞行为。解释这为什么是背压，而不是简单的性能问题。

第零部分

分布式计算：把单机原则扩展到集群

第 16 章

分布式计算模型

分布式计算把计算扩展到多台机器，但同时引入网络通信和故障。共享内存消失后，线程之间的读写变成消息；锁和原子不再直接可用；时间、顺序和失败都变得不可靠。

16.1 网络不是共享内存

单机内存访问通常以纳秒计，网络往返通常以微秒到毫秒计。单机 cache miss 已经很贵，跨网络请求更贵。分布式系统要尽量减少同步往返，使用批处理、流水线、数据本地性和异步通信。

网络还可能丢包、重排、延迟抖动或断开。应用层必须处理超时、重试、重复请求和部分失败。一个请求超时，不代表对方没有执行；重试可能造成重复副作用，因此幂等性非常重要。

16.2 RPC 的成本

RPC 把远程调用包装成类似本地函数调用的形式，但它不是本地函数。参数要序列化，消息要进入内核或用户态网络栈，经过网卡和交换机，到达对端后再反序列化和调度执行。一次 RPC 的固定成本远高于一次函数调用，因此分布式系统要避免细粒度频繁 RPC。

批处理、流水线和异步调用是常见优化。批处理减少每条消息固定开销，流水线隐藏往返延迟，异步调用避免线程阻塞。但它们都会增加错误处理复杂度：部分成功、乱序返回、超时重试和取消都要定义清楚。

RPC 还会隐藏背压问题。本地函数调用如果慢，调用者自然等在当前线程；远程调用如果被异步提交，调用者可能继续制造更多请求，直到队列、连接池或对端服务被压垮。因此 RPC 客户端必须有并发上限、超时、重试预算和排队策略。没有这些边界，异步 RPC 只是把阻塞变成不可控积压。

16.3 时间和顺序

不同机器没有完美同步时钟。日志时间戳可以帮助观察，但不能单独作为一致性依据。分布式系统常用逻辑时钟、版本号、任期号和日志索引表达顺序关系。

顺序要区分因果顺序和物理时间顺序。请求 A 触发请求 B，B 因果上发生在 A 之后；但两个机器的墙钟时间可能显示 B 的时间戳更早。协议通常不依赖墙钟证明因果，而是使用请求 ID、版本号、日志索引、任期或向量时钟等逻辑信息。日志时间戳主要用于观测和排查，不应被误用成强一致证据。

16.4 幂等性

幂等操作执行一次和执行多次效果相同。分布式系统中，客户端超时后常常不知道服务器是否执行了请求。若请求不是幂等的，重试可能造成重复扣款、重复创建或重复写入。常见做法是给请求分配唯一 ID，服务器记录已处理 ID，并在重复请求时返回同一结果。

幂等设计要落到存储结构。只在客户端生成请求 ID 不够，服务器还要把请求 ID、执行结果和业务状态放在同一个原子提交边界里。否则服务器可能已经修改业务状态，但还没记录请求 ID 就崩溃，重试后仍然重复执行。工程上常用去重表、条件写、事务日志或状态机日志解决这个问题。

16.5 故障模型

机器可能宕机，进程可能重启，网络可能分区，磁盘可能慢，消息可能重复。多数工程系统采用 crash-stop 或 crash-recovery 模型，不处理任意恶意错误。明确故障模型，是协议设计的第一步。

16.6 数据本地性

分布式计算中，移动计算到数据附近通常比移动大量数据更划算。MapReduce、Spark、分布式数据库和对象存储都围绕数据分片、任务调度和网络 shuffle 做权衡。

数据本地性是 NUMA 原则在集群尺度上的版本。单机里远端 NUMA 访问慢，集群里跨机拉取数据更慢；单机里先在线程本地 partial 再合并，集群里先在 map 端 combine 再 shuffle；单机里热点 cache line 会拖慢所有核心，集群里热点 key 会拖慢整个 stage。前面章节的思想在这里不是类比游戏，而是同一原则换了成本单位。

16.7 CAP 的正确理解

CAP 不是说系统只能在一致性、可用性、分区容忍性里任选两个。网络分区是分布式系统必须面对的现实；在分区发生时，系统需要在继续响应请求和保持强一致之间做选择。正常情况下，系统仍然要同时追求低延迟、高可用和足够一致。工程讨论要具体到故障场景，而不是只背缩写。

16.8 超时、重试和重试风暴

超时不是失败证明，只是本地等待时间超过阈值。请求可能在网络上，可能在对端队列里，可能已经执行但响应丢失，也可能确实失败。重试可以提高成功率，也可能放大过载。如果一个服务已经慢了，所有客户端同时重试，会制造重试风暴，让系统更慢。

健康的重试策略通常包括指数退避、抖动、最大重试次数、总超时预算、幂等保证和熔断。重试预算尤其重要：一个上游请求如果调用十个下游，每个下游都重试三次，总请求量可能迅速膨胀。分布式系统的性能问题常常来自这些控制路径，而不是用户函数本身。

核心思想

分布式系统的性能问题往往是正确性问题的影子。重试、复制、超时、共识和恢复都会改变吞吐和延迟。

实验：在本机用多个进程模拟客户端和服务端。人为加入随机延迟和超时，观察重试如何造成重复请求。设计一个幂等请求 ID 机制。

第 17 章

分片、复制与共识

分布式系统通过分片扩展容量，通过复制提高可用性，通过共识在故障中维护关键状态一致。三者共同构成大多数存储和计算平台的基础。

17.1 分片

分片把数据按 key、范围或哈希分到不同节点。好的分片要负载均衡、支持扩容、减少跨分片事务。热点 key 会破坏均衡，需要拆分、缓存、限流或特殊路由。

分片后的查询可能需要 scatter-gather：向多个分片发请求，再合并结果。它类似并行归约，但网络延迟和节点故障让合并更复杂。

分片策略要匹配访问模式。哈希分片适合点查和均匀 key，范围分片适合范围扫描和排序，但容易出现热点区间。虚拟分片把逻辑分片数量做得比物理节点多，便于迁移和均衡。热点 key 不能只靠增加节点解决，因为所有请求仍然打到同一个逻辑位置；需要缓存、拆 key、限流、异步化或改变业务模型。

17.2 再分片

系统运行一段时间后，数据量和热点会变化，需要再分片。再分片要迁移数据、更新路由、处理迁移期间的读写，并保证失败后可恢复。简单哈希分片在节点数变化时可能移动大量 key，一致性哈希或虚拟分片可以减少迁移范围。

再分片最难的是迁移期间的一致性。客户端可能按旧路由写，控制面可能已经发布新路由，数据可能正在复制。常见做法是让分片有状态机：准备迁移、双写或转发、追平增量、切换路由、清理旧副本。每一步都要能失败重试。没有状态机的再分片脚本，往往在中途失败后留下难以解释的双写和漏写。

17.3 复制

复制保存多个副本，提高可用性和读吞吐。同步复制延迟高但数据更安全，异步复制延迟低但可能丢失最新写入。主从复制简单，multi-leader 和 leaderless 复制更复杂，需要处理冲突。

复制协议必须说明写入何时算成功。如果 leader 写本地日志就返回，延迟低但 leader 崩溃可能丢数据；如果多数副本确认后返回，延迟高但故障恢复更稳；如果所有副本确认后返回，可用性和尾延迟都受最慢副本影响。工程系统通常让不同数据选择不同级别：元数据强一些，缓存和统计弱一些。

17.4 读写仲裁

Quorum 系统常用 W 个写副本和 R 个读副本满足 $R+W>N$ ，从而让读集合和写集合有交集。这个模型帮助理解一致性和可用性权衡。 W 大提高写安全性但增加写延迟， R 大提高读新鲜度但增加读延迟。实际系统还要处理慢副本、hinted handoff、read repair 和版本冲突。

17.5 一致性

强一致性让读者看到最新写入，但通常需要更多同步。最终一致性允许副本暂时不同，换取可用性和低延迟。选择一致性模型要看业务语义：计费、库存和元数据通常更偏强一致；缓存、推荐和日志统计可以接受延迟收敛。

17.6 共识

Raft 和 Paxos 这类共识协议用于在故障中让多个节点就日志顺序达成一致。共识不是让所有操作都变快，而是让关键状态在故障中仍然可恢复。共识路径应尽量小而清晰，不应把所有大数据流

量都压进共识日志。

共识适合维护小而关键的顺序状态，例如元数据、配置、leader 选举、分片归属和事务提交点。把大对象、热数据流和高频统计全部放进共识日志，通常会让系统吞吐受限。高性能分布式系统常把控制面和数据面分开：控制面用共识保证关键决策一致，数据面用分片、复制、批处理和校验保证吞吐。

17.7 Leader、任期和日志

以 Raft 的直觉看，系统在一个任期内选出 leader，客户端写入先进入 leader 日志，leader 复制给 follower，多数确认后提交。任期号防止旧 leader 在网络恢复后继续破坏新状态，日志索引和任期共同描述顺序。理解这些概念，能帮助读者读懂很多分布式存储的元数据路径。

17.8 读路径和租约

写路径需要顺序，读路径也要定义新鲜度。强一致读可能需要联系 leader 或多数副本，延迟较高；从 follower 读延迟低，但可能读到旧数据；租约读通过时间约束减少通信，但依赖时钟和租约安全边界。选择读路径时要问：业务是否允许旧读，旧到什么程度，故障切换时如何避免旧 leader 继续服务强一致读。

17.9 复制和备份

复制和备份经常被混为一谈。复制解决的是在线可用性：某个副本不可用时，系统还能从其他副本继续服务。备份解决的是历史恢复：数据被误删、程序 bug 写坏、勒索软件破坏或运维误操作后，系统能回到过去某个正确版本。复制通常追求低延迟和快速同步，备份通常追求隔离、保留周期和可验证恢复。一个系统可以有三个副本，仍然必须有备份。

常见误区

不要把复制等同于备份。复制会快速传播错误删除和错误写入；备份强调历史恢复。两者解决的问题不同。

实验：设计一个三副本 key-value 存储的写入流程。分别画出同步多数写、异步主从写和读修复流程，并说明各自的延迟和失败行为。

第 18 章

分布式计算工程实践

分布式计算工程把算法、运行时、存储、网络 and 观测性结合起来。真正的系统不仅要在正常路径跑得快，还要在机器慢、网络抖动、任务失败和数据倾斜时保持可诊断、可恢复。

18.1 Shuffle

Shuffle 把数据按 key 重新分布，是很多分布式计算框架的核心成本。它包含序列化、压缩、网络传输、落盘、排序、合并和反序列化。一个 group-by 或 join 的成本，常常主要来自 shuffle，而不是用户函数。

优化 shuffle 的方向包括减少数据量、提前聚合、压缩、分区均衡、避免热点 key、使用本地磁盘顺序写和控制并发拉取。它是分布式版的数据布局问题。

Shuffle 的基本流程可以拆成 map 端分区、局部排序或缓冲、落盘、网络拉取、reduce 端合并。每一步都有独立瓶颈：序列化消耗 CPU，压缩节省网络但增加计算，落盘受磁盘带宽影响，拉取并发过高会压垮对端，key 倾斜会让少数 reduce 任务变成长尾。优化 shuffle 不能只调一个参数，要先确认慢在哪一步。

18.2 检查点和容错

长任务必须考虑失败。检查点保存中间状态，让失败后从最近点恢复，而不是从头开始。检查点太频繁会拖慢正常路径，太稀疏会增加恢复成本。流式计算还要处理 exactly-once 或 at-least-once 语义。

检查点设计要说明一致性边界。批处理任务可以在 stage 边界保存输出，失败后重跑某个分区；流式任务需要同时保存输入 offset、算子状态和输出提交位置。若只保存算子状态，不保存已经输出到哪里，恢复后可能重复输出；若只保存 offset，不保存状态，聚合结果会丢失。所谓 exactly-once 往往不是“消息真的只执行一次”，而是通过幂等提交、事务输出或去重让外部效果等价于一次。

18.3 观测性

没有观测性，分布式系统无法维护。日志、指标、trace、profile 和事件审计要能回答：请求经过哪些节点，在哪里等待，重试了几次，队列多长，哪个分片慢，哪个任务倾斜，哪次部署引入变化。

观测性要围绕问题设计。延迟问题需要端到端 trace 和每段耗时；吞吐问题需要输入速率、处理速率和队列长度；倾斜问题需要按 key、分片、worker 的分布；可靠性问题需要错误码、重试、超时和恢复路径；资源问题需要 CPU、内存、磁盘、网络 and GC 或分配器指标。只打印一行“任务失败”没有工程价值。

18.4 从单机原则到集群原则

第二册前面讲的原则在分布式系统中仍然成立。局部性对应数据本地调度；锁竞争对应热点 key；false sharing 对应多个任务争用同一分片；背压对应限流和队列控制；归约对应 map-side combine 和树形聚合；缓存一致性对应复制一致性和失效策略。系统规模变大，原则没有消失，只是成本单位从 cache line 变成网络消息和磁盘块。

18.5 一个贯穿工程案例

考虑一个日志统计系统：输入日志按文件分片放在多台机器上，每个 worker 读取本地分片，解析出 key，先做本地 hash 聚合，再把 partial 按 key 分区 shuffle 到 reduce worker，最后合并并写出结果。这个系统把第二册几乎所有原则串在一起。本地解析要考虑 CPU 和内存布局，本地聚合要避免锁热点，shuffle 要处理网络和背压，reduce 要处理 key 倾斜，输出要保证幂等，失败后要能从检查点恢复。

如果某个 key 特别热，单机里它像一个热点锁，分布式里它像一个热点分片。解决方法也相似：先把热点拆成多个局部 partial，再在后续阶段合并。若网络拉取过多，像单机任务队列无背压一样，系统会排队爆炸。若 worker 失败后重跑，输出必须用任务 ID 和分区 ID 保证幂等提交。这样的案例比孤立讲术语更能训练系统思维。

核心思想

分布式计算不是单机计算的替代，而是单机计算原则在通信、故障和复制约束下的扩展。

综合实验：设计一个小型分布式日志统计系统。输入按文件分片，worker 本地统计，shuffle 按 key 聚合，driver 合并结果。写清数据格式、失败重试、幂等输出、指标和瓶颈。

附录 A

实验路线

第二册的实验围绕 Compute Systems Labs 展开。第一组实验是硬件与测量基础，包括 `lscpu` 拓扑记录、缓存层级观察、TLB 与缺页计数、`perf stat`、顺序归约和并行归约。第二组实验是数据布局，包括 AoS、SoA、false sharing、顺序访问、随机访问和预取效果。第三组实验是单机高性能计算，包括多累加器、SIMD 自动向量化、Roofline 分析和典型内核模式。

第四组实验是多线程同步，包括 mutex counter、atomic counter、bounded queue、条件变量、信号量和线程亲和性。第五组实验是无锁结构，包括 SPSC ring buffer、CAS 循环、ABA 现象、版本标签和内存回收方案分析。第六组实验是并行算法，包括 reduce、scan、partition、thread-local histogram 和任务粒度比较。第七组实验是运行时，包括线程池、任务队列、工作窃取模型、异步 I/O 和背压。第八组实验是分布式计算，包括本机多进程 RPC 模拟、超时重试、幂等请求、分片、复制和 shuffle。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件、futex、epoll、内存分配和网络栈。阅读时不要从全局搜索函数名开始，而要围绕问题进入：线程为什么迁移、缺页为什么发生、futex 怎样把用户态锁和内核等待连接起来。

glibc 和 musl 用于阅读 pthread、mutex、condition variable、malloc 和系统调用封装。重点观察用户态 fast path 和进入内核 slow path 的分界。

Intel TBB、oneTBB 或 LLVM OpenMP runtime 用于阅读任务调度、线程池、工作窃取和并行算法运行时。阅读时关注任务粒度、队列结构、窃取策略和 shutdown 语义。

Folly、Abseil 和 Boost.Lockfree 用于阅读并发容器、原子封装、hazard pointer、RCU 风格回收和高性能队列。阅读时先搞清进度保证和内存回收，不要只看 CAS。

Redis、RocksDB、Kafka 和 Spark 用于阅读分布式计算和存储工程：事件循环、后台线程、LSM compaction、分区、复制、日志、shuffle、背压和故障恢复。阅读时抓数据流和失败路径，不要只列模块名。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

乱序执行 CPU 在保持单线程可观察结果不变的前提下，提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径，预测失败会清空错误路径上的工作。

cache line 缓存一致性和缓存填充的基本粒度，常见大小为 64 字节。

TLB 地址翻译缓存，用于缓存虚拟页到物理页的映射。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

false sharing 多个线程写不同变量，但变量落在同一 cache line 上，导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束，例如 relaxed、acquire、release 和 seq_cst。

lock-free 一类进度保证，表示系统整体能持续前进，但不保证每个线程有限步完成。

背压 下游处理不过来时向上游施加的流量控制机制。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。

索引

instruction

add, 7